

Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет информационных технологий и управления

Кафедра систем управления

К защите допустить:

Исполняющий
обязанности заведующего
кафедрой информатики
_____С.И.Сиротко

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
к дипломному проекту
на тему

**ПРОГРАММНЫЙ КОМПЛЕКС ДЛЯ ПОДГОТОВКИ К
ЦЕНТРАЛИЗОВАННОМУ ТЕСТИРОВАНИЮ И
ЦЕНТРАЛИЗОВАННОМУ ЭКЗАМЕНУ**

БГУИР ДП 1-40 04 01 00 007 ПЗ

Студент	Б.Я.Дмитрук
Руководитель	С.В.Снисаренко
Консультанты:	
<i>от кафедры Информатики</i>	Н.Ю.Гриценко
<i>по экономической части</i>	О.А.Матяс
Нормоконтролер	Н.Ю.Гриценко
Рецензент	Н.Ю.Гриценко

Минск 2026

Решением рабочей комиссии
допущен(а) к защите дипломного проекта

Председатель рабочей комиссии
____ (_____)
Подпись Инициалы и фамилия

«___» июня 2026 г.

Министерство образования Республики Беларусь
Учреждение образования
«Белорусский государственный университет
информатики и радиоэлектроники»

Номер зачетной книжки 25350042

Практика зачтена с оценкой

_____ (_____)
(цифрой) (прописью)

(подпись руководителя практики от БГУИР)
_____.2026

ОТЧЕТ

по практике

Место прохождения практики: ООО «ПЛЮС ТДИ», г.Минск
Сроки прохождения практики: с 09.02.2026 по 22.03.2026

Руководитель практики от
предприятия:

_____ Г.А.Тылиндус
(подпись руководителя)
М.П.

Студент группы 253504

_____ Б.Я.Дмитрук
(подпись студента)

Руководитель практики от БГУИР
Снисаренко С.В. — старший
преподаватель кафедры
информатики

СОДЕРЖАНИЕ

Введение.....	6
1 Обзор и анализ существующих решений	8
1.1 Онлайн-платформы подготовки к экзаменам	8
1.2 Мобильные приложения.....	11
1.3 Достоинства и недостатки существующих решений	14
1.4 Выводы из анализа и постановка задач для разработки	15
2 Требования к программному комплексу	18
2.1 Общая постановка задачи.....	18
2.2 Функциональные требования.....	18
2.3 Нефункциональные требования	22
2.4 Требования к интерфейсам	23
2.5 Требования к безопасности.....	24
3 Проектирование программного комплекса	26
3.1 Архитектурные решения	26
3.2 Модульная структура системы	28
3.3 Взаимодействие компонентов	31
3.4 Сценарии использования системы	35
3.5 Обеспечение расширяемости и сопровождения	49
4 Проектирование и разработка базы данных.....	50
4.1 Обоснование выбора СУБД	50
4.2 Логическая модель данных	50
4.3 Физическая модель данных.....	56
5 Программная реализация.....	62
5.1 Выбор средств и инструментов разработки	62
5.2 Структуре серверной части	63
5.3 Структура мобильного приложения.....	65
5.4 Структура веб-инструмента преподавателя	67
5.5 Структура административной панели.....	70
5.6 Поддерживаемые платформы и окружения	70
6 Экономическое обоснование разработки и реализации программного комплекса на массовом рынке	72
6.1 Характеристика программного средства	Ошибка! Закладка не определена.

6.2 Расчет инвестиций в разработку программного средства	Ошибка!
Закладка не определена.	
6.3 Расчет экономического эффекта от реализации программного средства на рынке	Ошибка! Закладка не определена.
6.4 Расчет показателей экономической эффективности разработки и реализации программного средства на рынке	Ошибка! Закладка не определена.
Заключение	82
Список литературных источников	84
Приложение А Справка о проверке на взаимствования	85
Приложение Б Листинг программного кода	86
Приложение В Графический интерфейс пользователя	90
Приложение Г Use-case диаграмма программного средства.....	92
Приложение Д Entity–relationship диаграмма программного средства	93
Приложение Е Схема алгоритма обработки данных.....	69

ВВЕДЕНИЕ

Система образования в Республике Беларусь характеризуется последовательным развитием в направлении стандартизации и цифровизации. Ключевым этапом в образовательной траектории выпускников средней школы является прохождение централизованного тестирования, а в последние годы – централизованного экзамена, который выступает основным инструментом отбора абитуриентов в учреждения высшего образования. Успешность сдачи данного экзамена определяет возможность продолжения образования, что предопределяет высокую значимость вопросов организации эффективной подготовки.

Существующие традиционные средства подготовки включают использование печатных сборников тестов и индивидуальные занятия с преподавателями. Однако данные подходы имеют ограничения. Печатные материалы не обеспечивают адаптивности, отсутствуют средства систематического анализа ошибок и построения статистики результатов. Занятия с репетитором позволяют восполнять пробелы в знаниях, но требуют значительных затрат и не обладают массовой доступностью. В условиях развития информационных технологий и широкого распространения мобильных устройств актуальным становится использование специализированных цифровых инструментов, которые способны обеспечить доступность, гибкость и регулярность подготовки.

На данный момент на рынке образовательных решений присутствует ряд приложений, ориентированных на подготовку к международным экзаменам и отдельным предметным дисциплинам. Однако данные решения, как правило, не учитывают специфику белорусской системы итоговой аттестации и не предоставляют средств взаимодействия между учеником и преподавателем. В большинстве случаев функциональность ограничивается представлением набора готовых тестов без возможности их комбинирования, анализа динамики подготовки или формирования индивидуальных подборок заданий. Данный факт обосновывает необходимость разработки программного комплекса, ориентированного на специфику национальной системы образования.

Целью настоящей работы является разработка программного комплекса, включающего мобильное приложение для операционной системы Android, серверную часть на платформе .NET и административную панель на базе Django, предназначенного для организации подготовки выпускников к централизованному экзамену. Разрабатываемое решение предусматривает хранение и предоставление тестов прошлых лет, возможность формирования смешанных и пользовательских тестов, сбор и анализ статистики решений, формирование рекомендаций по повторению тем, а также поддержку роли

преподавателя, включающей создание собственных тестов, контроль прогресса учеников и обмен материалами.

Для достижения указанной цели решаются следующие задачи: организация базы тестовых заданий и её структурирование по предметным областям и типам заданий; реализация механизмов адаптивного формирования тестов; разработка подсистемы статистики и аналитики для учащихся и преподавателей; обеспечение функциональности взаимодействия между пользователями с различными ролями; реализация механизмов защиты и безопасного хранения данных.

Практическая значимость разработки заключается в создании доступного цифрового инструмента, обеспечивающего повышение эффективности подготовки к централизованному экзамену. В отличие от существующих аналогов, предлагаемый программный комплекс ориентирован на специфику белорусской системы образования и сочетает функции самостоятельной подготовки учащихся и контроля со стороны преподавателя. Использование комплекса позволит учащимся систематизировать знания и объективно оценивать динамику подготовки, а преподавателям – эффективно организовывать образовательный процесс и осуществлять мониторинг результатов.

1 Обзор и анализ существующих решений

1.1 Онлайн-платформы подготовки к экзаменам

1.1.1 Онлайн-платформа Adukar.

Одним из наиболее известных белорусских ресурсов для подготовки к централизованному тестированию и централизованному экзамену является платформа Adukar. Сайт функционирует в формате образовательного портала и предоставляет учащимся широкий спектр инструментов для подготовки к поступлению в высшие учебные заведения. Основная функциональность платформы сосредоточена вокруг онлайн-решения тестов по различным предметам, а также предоставления методических материалов и статей, связанных с подготовкой к экзаменам.

Ключевой возможностью Adukar является доступ к реальным вариантам тестов прошлых лет. Учащиеся могут выбрать интересующий их предмет, ознакомиться с форматом заданий и пройти тест в онлайн-режиме. Система фиксирует количество правильных и неправильных ответов, что позволяет пользователю получить общее представление о своём уровне подготовки. Однако результаты представляются преимущественно в виде подсчёта правильных ответов без глубокой аналитики и детальной статистики по тематическим разделам.

Дополнительно платформа предоставляет справочные материалы, объяснения по типовым ошибкам, а также статьи о стратегии прохождения экзаменов. Это расширяет возможности ресурса, однако в большинстве случаев информация подаётся в статическом виде и не интегрирована в адаптивную систему обучения. Таким образом, учащийся получает полезные сведения, но вынужден самостоятельно выстраивать стратегию повторения тем и организации подготовки.

Сильной стороной Adukar является высокая доступность и популярность среди школьников. Также положительным фактором является соответствие тестов формату централизованного экзамена, что позволяет учащимся заранее привыкнуть к структуре заданий и регламенту.

К ограничениям платформы можно отнести отсутствие персонализации и адаптивного подхода к обучению. Система не предоставляет индивидуальных рекомендаций, не формирует подборки заданий исходя из ошибок пользователя, а также не имеет развитых инструментов для анализа динамики подготовки. Кроме того, отсутствует функциональность взаимодействия с преподавателем, что делает невозможным использование платформы в рамках полноценного образовательного процесса под руководством наставника.

В отличие от Adukar, разрабатываемый в рамках данного проекта программный комплекс предусматривает интеграцию механизмов адаптивного формирования тестов и детальной статистики по темам и категориям заданий. Пользователь сможет не только увидеть процент

правильных ответов, но и получить рекомендации по темам, требующим повторения. Также в системе реализуется функциональность для преподавателей, которые смогут создавать собственные тесты, отслеживать прогресс учеников и управлять учебным процессом. Это обеспечивает более высокий уровень гибкости и эффективности по сравнению с существующими возможностями платформы Adukar.

1.1.2 Онлайн-платформа TurboСТ.

Среди белорусских онлайн-ресурсов, ориентированных на подготовку к централизованному тестированию и централизованному экзамену, значительное распространение получила платформа TurboСТ. Данный сервис специализируется на предоставлении пользователям доступа к базе реальных тестов прошлых лет и тренировочных заданий в формате, максимально приближенном к официальным экзаменационным условиям. Основная идея платформы заключается в том, чтобы воспроизвести режим централизованного экзамена в онлайн-среде и тем самым подготовить учащихся к реальной ситуации прохождения испытаний.

Главная функциональная возможность TurboСТ заключается в прохождении полных вариантов тестов с учётом временных ограничений. Пользователь выбирает предмет, запускает тест и должен выполнить его в установленное время. По завершении работы система автоматически подсчитывает результат и переводит его в экзаменационную шкалу. Такой подход позволяет школьникам заранее привыкнуть к регламенту экзамена и тренировать навык распределения времени на выполнение заданий. Это отличает TurboСТ от некоторых других ресурсов, которые предоставляют тесты без имитации временных условий.

Сервис также предоставляет пользователям доступ к архиву тестов прошлых лет, что позволяет систематически отрабатывать типовые задания. После завершения работы можно просмотреть количество правильных и неправильных ответов, получить балл и сравнить результат с предыдущими попытками. Однако, несмотря на наличие базовой статистики, система не предусматривает детализированного анализа по тематическим разделам и не формирует индивидуальных рекомендаций для повторения материала.

К достоинствам TurboСТ можно отнести простоту использования, ориентацию именно на белорусский формат экзамена и имитацию реальной экзаменационной обстановки. Эти факторы позволяют пользователям не только проверить уровень знаний, но и психологически подготовиться к процедуре экзамена. Дополнительным преимуществом является широкая доступность: платформа функционирует в веб-браузере и не требует установки дополнительных программ.

К числу ограничений TurboСТ относится отсутствие расширенной системы аналитики, невозможность комбинирования заданий из разных тестов, а также ограниченная поддержка взаимодействия между учащимся и преподавателем. Платформа не предусматривает роли учителя, который мог

бы формировать собственные варианты заданий или отслеживать прогресс прикрепленных учеников. Кроме того, отсутствует механизм адаптивного формирования тестов, что снижает гибкость индивидуальной подготовки.

Разрабатываемый программный комплекс преодолевает обозначенные ограничения. В частности, в нём реализуется возможность генерации смешанных тестов из заданий разных лет и разных категорий, что позволяет создавать уникальные варианты, ориентированные на отработку конкретных тем. Дополнительно будет обеспечена функция персонализированных рекомендаций и подробная статистика по результатам прохождения тестов. Наличие преподавательского интерфейса позволит интегрировать самостоятельную подготовку учащегося с контролем и сопровождением со стороны педагога, что существенно расширяет сферу применения системы по сравнению с возможностями TurboСТ.

1.1.3 Онлайн-платформа 0101СТ.

Среди белорусских образовательных ресурсов, ориентированных на подготовку к централизованному тестированию и централизованному экзамену, выделяется платформа 0101СТ. В отличие от большинства аналогичных решений, данный проект позиционирует себя не только как база тестов, но и как комплексная система обучения, включающая онлайн-курсы, методические материалы и инструменты для работы с преподавателями.

Основной функционал платформы включает прохождение тестов по предметам централизованного экзамена, доступ к курсам и выполнение домашних заданий. Пользователь получает возможность работать как в веб-интерфейсе, так и через мобильное приложение. Сервис реализует систему тематической структуризации материалов: тестовые задания и учебные блоки сгруппированы по разделам, что позволяет последовательно осваивать материал. Дополнительным элементом является интеграция домашних заданий и проверочных работ, которые направляются ученикам в рамках курсовой программы.

Сильной стороной 0101СТ является комплексный подход к подготовке. Платформа сочетает тестирование с теоретическими материалами, а также поддерживает элемент взаимодействия с преподавателем. Такой формат приближает её к онлайн-школам, где учебный процесс выстраивается не только вокруг самопроверки, но и в виде последовательного освоения программы. Кроме того, наличие мобильного приложения обеспечивает удобство доступа к материалам и позволяет учащимся использовать сервис в любое время.

Однако при всех преимуществах у платформы есть ряд ограничений. Во-первых, доступ к расширенным возможностям, включая курсы и проверку домашних заданий, как правило, является платным, что снижает массовую доступность ресурса. Во-вторых, система не предусматривает свободного комбинирования заданий по выбору ученика, а учебный процесс во многом строится по фиксированным сценариям. В-третьих, несмотря на наличие

взаимодействия с преподавателями в рамках курсов, отсутствует полноценная система персонализированной аналитики, которая могла бы автоматически выявлять слабые места и формировать рекомендации.

Разрабатываемый программный комплекс отличается от 0101СТ прежде всего ориентацией на массовое использование и универсальность. В нём будет реализован механизм гибкой генерации тестов из различных источников, возможность создания кастомных заданий и тестов преподавателями, а также система рекомендаций для учащихся. Такой подход позволит совместить преимущества формального курса и свободной тренировки, обеспечив при этом доступность функционала. Дополнительно в проекте предусмотрена расширенная статистика и аналитика, что обеспечит более высокий уровень персонализации, чем в существующем решении.

1.2 Мобильные приложения

1.2.1 Мобильное приложение Adukar.

Мобильное приложение Adukar является продолжением одноимённой образовательной платформы и предназначено для подготовки школьников к централизованному тестированию и централизованному экзамену. Оно предоставляет доступ к базе тестовых заданий прошлых лет, а также дополнительным материалам, представленным на основном веб-ресурсе. Приложение ориентировано на операционную систему Android и распространяется в свободном доступе.

Ключевая функциональность мобильного приложения заключается в возможности прохождения тестов по выбранным предметам. Пользователь получает набор заданий, близкий по структуре к официальным экзаменационным материалам, и может пройти тестирование в режиме тренировки. По итогам прохождения выводится количество правильных ответов и процент успешности. Кроме того, приложение содержит справочные материалы и рекомендации, что делает его более универсальным инструментом по сравнению с исключительно веб-версией.

Сильной стороной мобильного приложения Adukar является его доступность и удобство. Пользователь имеет возможность готовиться к экзамену в любое время, используя смартфон, что значительно расширяет сценарии применения ресурса. Дополнительным преимуществом является наличие контента, соответствующего официальному формату централизованного экзамена, что позволяет учащимся знакомиться с типовыми заданиями и вырабатывать навыки работы с ними в условиях, приближённых к экзаменационным.

В то же время у приложения наблюдается ряд ограничений. Во-первых, реализована преимущественно базовая функциональность: система фиксирует количество правильных ответов, но не предоставляет расширенной статистики по темам и типам заданий. Во-вторых, отсутствует механизм адаптивного подбора тестов, что не позволяет индивидуализировать процесс подготовки.

В-третьих, приложение не предусматривает возможности взаимодействия с преподавателем, отсутствует поддержка создания кастомных тестов и совместной работы в рамках учебного процесса. Таким образом, функциональность приложения ограничена самостоятельной подготовкой без интеграции в образовательную систему.

Разрабатываемый в рамках настоящего проекта программный комплекс преодолевает данные ограничения. В отличие от мобильного приложения Adukar, он будет включать подсистему аналитики и персонализации, позволяющую автоматически выявлять пробелы в знаниях и формировать рекомендации для повторения. Дополнительно предусмотрена функциональность генерации смешанных и пользовательских тестов, что повышает гибкость подготовки. Особое значение имеет поддержка роли преподавателя, который сможет создавать собственные тесты, отслеживать прогресс учеников и делиться материалами. Таким образом, предлагаемое решение обеспечивает более высокий уровень индивидуализации подготовки и интеграцию в образовательный процесс, что выводит его за рамки возможностей существующего мобильного приложения Adukar.

1.2.2 Мобильное приложение 0101СТ.

Мобильное приложение 0101СТ является составной частью одноимённого образовательного проекта, который предоставляет учащимся комплексные курсы подготовки к централизованному тестированию и централизованному экзамену. В отличие от ряда других решений, приложение не ограничивается функцией прохождения тестов: оно интегрировано в учебный процесс и ориентировано на взаимодействие между учеником и преподавателем.

Функциональность приложения включает доступ к тестам, домашним заданиям и теоретическим материалам, которые организованы в соответствии с курсами, предлагаемыми платформой. Пользователь получает задания в структурированном виде, что позволяет двигаться по программе последовательно, изучая материал по темам. Приложение поддерживает проверку домашних заданий преподавателем, что обеспечивает обратную связь и делает подготовку более системной. Важным элементом является уведомление пользователей о новых заданиях и результатах проверки, что позволяет поддерживать дисциплину подготовки.

Сильной стороной мобильного приложения 0101СТ является комплексность подхода и интеграция в образовательный процесс. Пользователь получает не только доступ к базе тестовых заданий, но и возможность обучаться по курсу под руководством наставника. Наличие мобильного клиента обеспечивает удобство доступа к материалам в любое время и с любого устройства, а встроенные механизмы контроля помогают выработать устойчивый график подготовки.

Вместе с тем у приложения имеются определённые ограничения. Доступ ко многим функциям, включая обучение в курсовом формате и проверку

домашних заданий, предоставляется на платной основе, что снижает массовую доступность ресурса. Структура заданий и последовательность их прохождения в значительной степени жёстко закреплены, что ограничивает гибкость подготовки. Кроме того, приложение не предоставляет расширенной статистики по результатам тестирования и не реализует адаптивные механизмы подбора заданий в зависимости от индивидуальных пробелов в знаниях.

Разрабатываемый в рамках настоящего проекта программный комплекс отличается от 0101СТ ориентацией на свободное использование и доступность основных функций. В нём предусмотрена возможность самостоятельного формирования тестов, а также генерации смешанных заданий по категориям и темам. Система будет включать детальную статистику и персонализированные рекомендации, чего в текущем решении нет. Важным отличием является функциональность преподавательского интерфейса, позволяющая педагогам создавать собственные тесты, прикреплять учеников и отслеживать их прогресс без необходимости жёсткой привязки к курсовой программе. Таким образом, предлагаемое решение сочетает преимущества индивидуальной и групповой подготовки при сохранении гибкости и доступности для пользователей.

1.2.3 Мобильное приложение Gojimo.

Одним из наиболее известных международных мобильных приложений для подготовки к экзаменам является Gojimo. Данный сервис ориентирован на учащихся Великобритании и США, готовящихся к экзаменам GCSE, A-Level, SAT и аналогичным. Приложение содержит обширную базу вопросов с несколькими вариантами ответов, сгруппированных по предметам и темам.

Ключевая функциональность заключается в возможности выбора предмета, прохождения теста и получения немедленного результата с указанием правильных и неправильных ответов. Приложение предоставляет объяснения к ряду вопросов, что помогает не только проверять знания, но и разбирать ошибки. Доступ к значительной части контента бесплатный, при этом некоторые материалы распространяются по подписке.

Сильными сторонами Gojimo являются большая база вопросов, охват различных экзаменационных систем и наличие пояснений к заданиям. Интерфейс оптимизирован для мобильных устройств, что обеспечивает удобство использования. Кроме того, приложение активно применяется в разных странах, что подтверждает его востребованность среди учащихся.

К ограничениям можно отнести ориентацию на международные экзамены, что делает данное приложение непригодным для прямого использования белорусскими абитуриентами. Структура заданий и база контента не соответствуют формату централизованного экзамена в Беларуси. Кроме того, функционал приложения ограничен прохождением тестов и базовой статистикой: отсутствует система рекомендаций, взаимодействие с преподавателем и адаптивная генерация тестов.

Разрабатываемый программный комплекс, в отличие от Gojimo, будет специально ориентирован на национальный экзаменационный формат и включать механизмы персонализации. Дополнительным преимуществом станет наличие преподавательского интерфейса, позволяющего создавать кастомные тесты и отслеживать прогресс учеников, что выводит проект за рамки возможностей существующих международных решений.

1.2.4 Мобильное приложение Examer.

Одним из приложений, ориентированных на подготовку к экзаменам в формате тестирования, является Examer. Оно позиционируется как универсальный инструмент для отработки экзаменационных заданий и поддерживает прохождение тестов по различным дисциплинам. Основная функциональность заключается в имитации условий реального экзамена: пользователь выбирает предмет, запускает тест и выполняет его в ограниченное время. По завершении система подсчитывает количество правильных ответов и формирует результат в виде общей оценки.

Сильной стороной Examer является простота интерфейса, возможность тренироваться в условиях, приближённых к экзаменационным, и поддержка нескольких предметов. Приложение ориентировано на массовое использование, доступно на мобильных устройствах и не требует сложных настроек.

К ограничениям можно отнести отсутствие развитой аналитики: система фиксирует лишь общий результат, без детализированного анализа по темам и категориям заданий. Кроме того, приложение не предусматривает адаптивного формирования тестов, отсутствует поддержка взаимодействия с преподавателем и создание кастомных вариантов заданий.

Разрабатываемый программный комплекс отличается от Examer более широким функционалом. В нём будет реализована система персонализированных рекомендаций, возможность генерации смешанных тестов по категориям, детальная статистика и роль преподавателя. Это обеспечивает более высокий уровень индивидуализации подготовки и интеграцию в образовательный процесс, что недоступно в рамках существующего мобильного приложения Examer.

1.3 Достоинства и недостатки существующих решений

Анализ онлайн-платформ и мобильных приложений, предназначенных для подготовки к централизованному тестированию и централизованному экзамену, позволяет выделить ряд общих достоинств, характерных для большинства рассмотренных решений. Прежде всего, данные ресурсы обеспечивают учащимся доступ к заданиям прошлых лет, что позволяет ознакомиться с форматом экзамена и типологией вопросов. Существенным преимуществом является высокая доступность: как онлайн-платформы, так и мобильные приложения могут использоваться на различных устройствах при

наличии подключения к сети Интернет, что расширяет возможности организации самостоятельной подготовки. Многие системы отличаются простотой интерфейса и низким порогом входа, что делает их удобными для массового применения. Для ряда ресурсов положительным фактором является наличие дополнительных материалов, включающих теоретические пояснения и методические рекомендации, что способствует более глубокому освоению содержания экзаменационных дисциплин.

Вместе с тем анализ продемонстрировал наличие существенных ограничений и недостатков, которые ограничивают эффективность применения существующих решений. Наиболее распространённым недостатком является отсутствие персонализированного подхода: большинство систем фиксируют лишь общее количество правильных и неправильных ответов, не предоставляя подробной аналитики по темам и категориям заданий. Практически полностью отсутствует функциональность адаптивного формирования тестов, позволяющая строить индивидуальные траектории подготовки. В большинстве случаев не реализована роль преподавателя: пользователи не имеют возможности прикрепляться к учителю, получать от него индивидуальные задания и рекомендации, а также предоставлять доступ к своей статистике. Кроме того, в существующих решениях недостаточно развита система мотивации: отсутствуют механизмы уведомлений, напоминаний и рекомендаций по повторению тем.

Таким образом, несмотря на очевидные достоинства — доступность, простоту и наличие базы экзаменационных заданий, — существующие платформы и приложения не обеспечивают комплексного решения задач подготовки к централизованному экзамену. Отсутствие адаптивности, глубокой аналитики и взаимодействия «учитель–ученик» предопределяет необходимость разработки нового программного комплекса, сочетающего в себе перечисленные возможности и ориентированного на специфику белорусской системы образования.

1.4 Выводы из анализа и постановка задач для разработки

Проведённый анализ существующих онлайн-платформ и мобильных приложений, ориентированных на подготовку к централизованному тестированию и централизованному экзамену, показал, что в настоящее время учащиеся располагают значительным количеством цифровых инструментов. Эти решения обладают рядом очевидных достоинств: они обеспечивают доступ к заданиям прошлых лет, позволяют знакомиться с форматом экзамена, отличаются высокой доступностью и простотой использования. Наличие мобильных приложений расширяет возможности подготовки за счёт использования смартфонов и планшетов, а интеграция с учебными курсами в отдельных проектах позволяет частично организовывать образовательный процесс под руководством преподавателя.

В то же время анализ показал и существенные недостатки, которые

ограничивают эффективность применения существующих решений. Большинство платформ и приложений предоставляют пользователю лишь набор готовых тестов без возможности гибкой генерации заданий. Системы статистики, как правило, ограничиваются фиксацией количества правильных и неправильных ответов без углублённой аналитики по темам и динамике подготовки. Практически полностью отсутствует персонализация, выражающаяся в автоматическом выявлении слабых мест и формировании индивидуальных рекомендаций. В большинстве рассмотренных решений не предусмотрено полноценное взаимодействие между учеником и преподавателем: отсутствует возможность создания кастомных тестов учителем, прикрепления учеников к аккаунту преподавателя и отслеживания их прогресса. Таким образом, существующие продукты частично закрывают потребности учащихся, но не обеспечивают комплексного подхода к подготовке, ориентированного на специфику централизованного экзамена в Беларуси.

Выявленные недостатки определяют постановку задач для разработки нового программного комплекса. В рамках дипломного проекта предполагается реализовать систему, которая объединяет преимущества существующих решений и устраняет их ключевые ограничения. Основными задачами разработки являются:

- создание базы тестов прошлых лет с возможностью их прохождения в режиме, приближенном к экзаменационному;
- реализация механизма генерации смешанных тестов, включающего выбор количества заданий по категориям и возможность интеграции кастомных тестов;
- разработка подсистемы статистики и аналитики, обеспечивающей детализацию по темам и динамике подготовки;
- внедрение системы персонализированных рекомендаций по повторению тем на основе результатов тестирования;
- реализация функциональности преподавательского интерфейса, включающей создание собственных тестов, прикрепление учеников и отслеживание их прогресса;
- обеспечение взаимодействия между учителем и учеником в рамках единой платформы;
- интеграция push-уведомлений и напоминаний для повышения мотивации и регулярности подготовки;
- разработка мобильного клиента для Android, обеспечивающего удобный доступ к функционалу и поддержку офлайн-режима.

Таким образом, на основании анализа существующих решений сформулирована задача разработки комплексного программного продукта, ориентированного на подготовку к централизованному экзамену в Беларуси и учитывающего современные требования к цифровым образовательным системам.

2 Требования к программному комплексу

2.1 Общая постановка задачи

Целью разработки является создание программного комплекса, обеспечивающего эффективную подготовку выпускников школ Республики Беларусь к централизованному экзамену посредством использования мобильного приложения, серверной части и административной панели. Комплекс должен включать функциональность как для самостоятельной подготовки учащихся, так и для организации учебного процесса под руководством преподавателей.

Основная задача программного комплекса заключается в предоставлении пользователю доступа к базе тестовых заданий прошлых лет и возможности формирования новых тестов, включающих как задания из официальных источников, так и созданные преподавателями. Система должна позволять учащемуся проходить тестирование в условиях, приближённых к экзаменационным, а также использовать режимы свободной тренировки. Результаты выполнения заданий должны сохраняться и представляться в виде статистики и аналитики, обеспечивающей оценку уровня подготовки и выявление пробелов в знаниях.

Программный комплекс должен обеспечивать персонализацию учебного процесса. На основании результатов прохождения тестов пользователю должны предоставляться рекомендации по повторению отдельных тем и разделов. Для повышения эффективности подготовки необходимо предусмотреть возможность формирования смешанных тестов, позволяющих комбинировать задания из разных источников и категорий.

Особое внимание должно уделяться роли преподавателя. В системе должен быть реализован отдельный интерфейс, позволяющий учителям создавать собственные тесты, назначать их прикреплённым ученикам, отслеживать результаты их выполнения и формировать рекомендации. Таким образом, комплекс должен обеспечить интеграцию индивидуальной и групповой подготовки в рамках единой цифровой среды.

С архитектурной точки зрения программный комплекс должен представлять собой распределённую систему, включающую мобильное приложение для операционной системы Android, серверную часть на платформе ASP.NET Core, административную панель для преподавателей, реализованную с использованием Django, а также базу данных PostgreSQL для централизованного хранения информации [1]. Обмен данными между компонентами должен осуществляться по протоколу HTTP с использованием REST API.

2.2 Функциональные требования

Следующие функциональные требования сформированы на основе

анализа существующих решений и задач, поставленных в настоящем проекте, и описывают поведение системы с точки зрения пользователей различных ролей.

1 Регистрация и аутентификация пользователей. Система должна обеспечивать возможность создания нового аккаунта с указанием минимального набора данных (имя, адрес электронной почты, пароль). Должны быть реализованы процедуры входа в систему, выхода и восстановления доступа в случае утраты пароля. Для корректной работы приложения необходимо предусмотреть уникальность учётных записей и защиту от повторной регистрации с уже существующими адресами электронной почты.

2 Роли пользователей и разграничение прав. В программном комплексе должны быть предусмотрены роли «ученик», «преподаватель» и «администратор». Роль «ученик» ограничивается функциональностью прохождения тестов, просмотра статистики и получения рекомендаций. Роль «преподаватель» дополнительно включает возможность создавать тесты, назначать их ученикам, просматривать их статистику и формировать аналитические отчёты. Роль «администратор» используется для управления банком заданий и пользователями через административную панель.

3 Механизм привязки учеников к преподавателю. Для обеспечения взаимодействия необходимо реализовать функциональность назначения учеников к определённому преподавателю. Должна быть предусмотрена возможность приглашения через уникальный код или подтверждение запроса со стороны ученика. После привязки преподаватель получает доступ к истории решений и статистике данного ученика.

4 Банк тестов прошлых лет. Система обязана содержать структурированную базу тестов, включающую все задания централизованного тестирования и экзамена прошлых лет. Для каждого задания должны храниться метаданные: предмет, год проведения, номер варианта, категория или тема, уровень сложности, правильный ответ и пояснение. Должен быть реализован поиск и фильтрация по указанным параметрам.

5 Прохождение готовых тестов прошлых лет. Пользователь с ролью «ученик» должен иметь возможность выбрать полный вариант теста, запустить его с таймером и пройти в условиях, соответствующих экзаменационным. По завершении выполнения система должна подсчитать количество правильных и неправильных ответов, отобразить результат и сохранить его в истории решений.

6 Формирование смешанных тестов. Приложение должно предоставлять пользователю возможность сформировать тест с заданным количеством вопросов по различным категориям и темам. Пользователь должен иметь возможность указать предмет, количество заданий каждой категории, уровень сложности, а также решить, будут ли включены задания, созданные преподавателем.

7 Создание пользовательских тестов преподавателями. Пользователь с

ролью «преподаватель» должен иметь возможность формировать тесты из заданий банка или добавлять новые вопросы. Должна быть предусмотрена возможность редактирования и удаления тестов, а также публикации их для прикрепленных учеников.

8 Создание пользовательских тестов. Преподаватель должен иметь возможность создавать собственные пользовательские тесты через специализированную браузерную панель и передавать эти тесты для выполнения выбранным ученикам. Ученик получает уведомление о новом тесте и может пройти его в своём приложении. Результаты выполнения должны сохраняться и быть доступны как ученику, так и преподавателю.

9 История решений тестов. Для каждого пользователя необходимо хранить список всех пройденных тестов. Запись должна содержать дату прохождения, количество заданий, результат в баллах, перечень правильных и неправильных ответов, а также время, затраченное на выполнение.

10 Аналитика и статистика. Система должна формировать статистику на основе накопленной истории решений. Для ученика статистика отображается в виде распределения ошибок по темам и динамики подготовки за выбранный период. Для преподавателя предоставляется доступ к агрегированной статистике по прикрепленным ученикам с возможностью просмотра индивидуальных данных.

11 Формирование рекомендаций. На основании анализа ошибок система должна автоматически определять темы, требующие повторения. Пользователь получает список тем, упорядоченных по приоритетности, а также уведомления с напоминанием о необходимости их повторить.

12 Система уведомлений. Программный комплекс должен поддерживать push-уведомления на мобильных устройствах. Уведомления используются для информирования о назначении новых тестов, появлении рекомендаций, обновлении результатов и напоминаниях о регулярной подготовке.

Реализация указанных функций позволит обеспечить комплексный подход к подготовке к централизованному экзамену в условиях современной цифровой образовательной среды.

В завершение описания функциональных требований к системе основные варианты использования и роли пользователей в наглядной форме представлены на диаграмме прецедентов на рисунке 2.1.

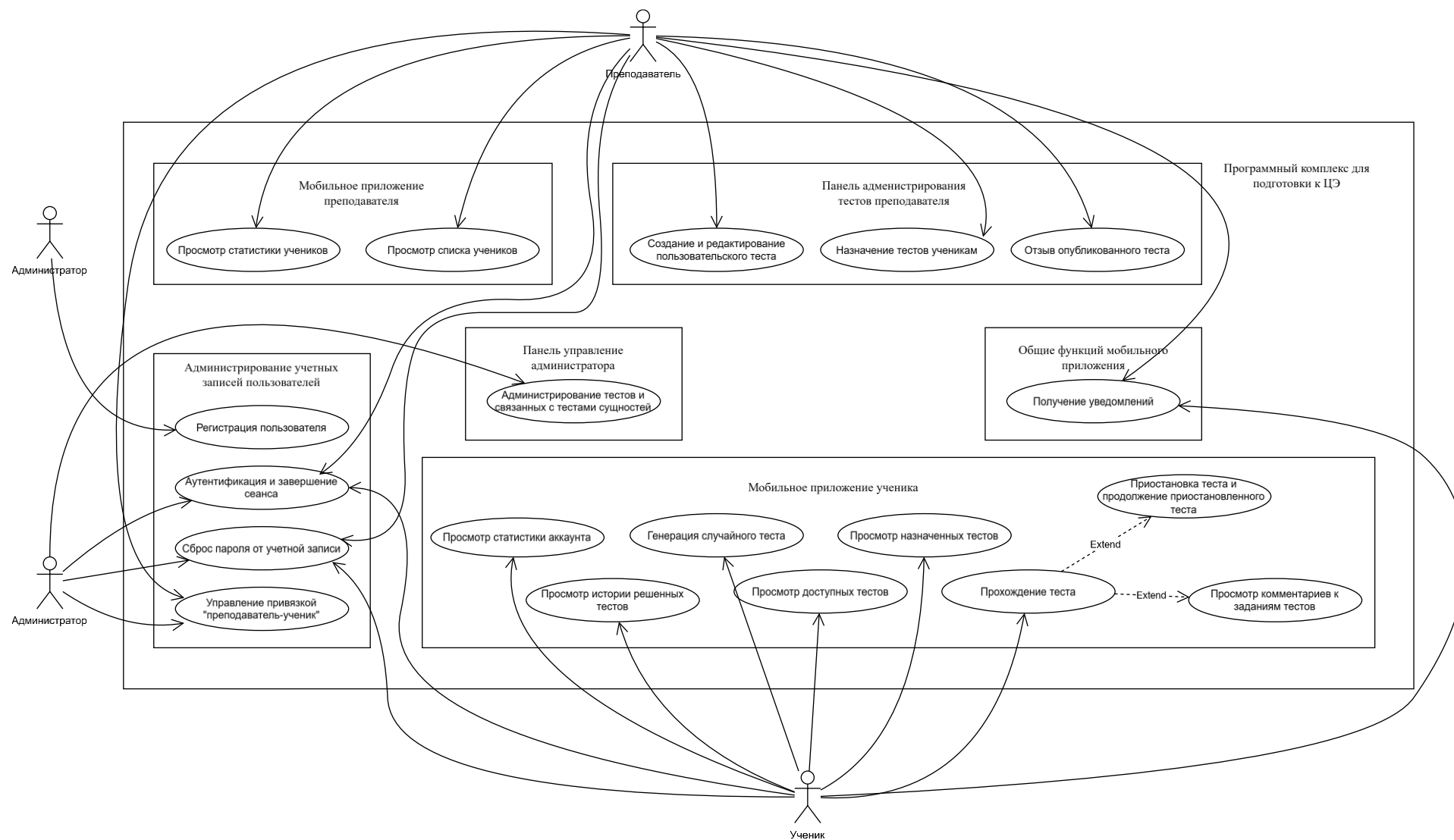


Рисунок 2.1 – Диаграмма прецедентов программного комплекса

2.3 Нефункциональные требования

С точки зрения производительности серверная часть на базе ASP.NET Core должна обеспечивать обработку стандартных запросов (получение списка тестов, отправка результатов попытки, загрузка статистики) за время не более одной секунды при одновременной работе не менее 500 активных пользователей [2]. Мобильное приложение на Kotlin должно обеспечивать мгновенный отклик интерфейса при переходах между основными экранами и отображать результаты теста не позднее чем через две секунды после отправки их на сервер.

Надёжность и отказоустойчивость являются важными характеристиками комплекса. В случае обрыва сетевого соединения мобильное приложение должно сохранять прогресс решения теста локально в SQLite и автоматически синхронизировать данные с сервером после восстановления связи. Результаты всех завершённых тестов должны храниться в базе данных PostgreSQL, где необходимо настроить ежедневное резервное копирование с возможностью восстановления не менее чем за последние семь дней.

Ключевым требованием является удобство интерфейсов. Мобильное приложение должно соответствовать гайдлайнам Material Design для Android, обеспечивая единообразие элементов и предсказуемость поведения интерфейса. Основной сценарий пользователя — выбор теста, его прохождение, просмотр результатов и статистики — должен выполняться не более чем в три шага. Веб-инструмент для преподавателей должен иметь адаптивный интерфейс на базе Django Templates, корректно отображаться в современных браузерах (Google Chrome, Mozilla Firefox, Microsoft Edge) и поддерживать разрешения экранов от 1366×768 пикселей.

Масштабируемость системы должна обеспечиваться за счёт использования модульной архитектуры серверной части и возможности горизонтального масштабирования базы данных PostgreSQL. Архитектура приложения должна позволять без доработок ядра добавлять новые предметы, категории и типы заданий. При росте нагрузки до 1000 активных пользователей система должна обеспечивать устойчивую работу без деградации производительности.

Совместимость комплекса должна гарантировать работу мобильного приложения на устройствах с Android версии 8.0 (Oreo) и выше. Серверная часть должна развёртываться на операционных системах Windows Server 2019 и Ubuntu 22.04 LTS. Для взаимодействия между компонентами должен использоваться протокол HTTPS, а обмен данными реализован в формате JSON поверх REST API.

Особое внимание уделяется безопасности. Для аутентификации и авторизации пользователей должны использоваться JWT-токены, генерируемые сервером ASP.NET Core и передаваемые в заголовках запросов. Пароли пользователей должны храниться только в виде хэшей,

сформированных с использованием алгоритма bcrypt с солью. Передача данных между клиентом и сервером должна осуществляться исключительно по протоколу HTTPS с использованием TLS 1.3. Система должна быть защищена от SQL-инъекций за счёт использования параметризованных запросов в Entity Framework Core, а также от атак XSS и CSRF средствами встроенных middleware. Доступ к функциям должен разграничиваться по ролям «ученик» и «преподаватель».

Поддержка и сопровождение комплекса должны обеспечиваться за счёт документирования API с использованием Swagger/OpenAPI, наличия юнит-тестов для основных сервисов на уровне серверной части и возможности развёртывания обновлений без потери пользовательских данных. В системе должно быть реализовано логирование ключевых операций с использованием библиотеки Serilog.

2.4 Требования к интерфейсам

2.4.1 Пользовательские интерфейсы.

Мобильное приложение для операционной системы Android должно быть ориентировано на пользователя с ролью ученика. Интерфейс обязан соответствовать рекомендациям Material Design и обеспечивать единообразие элементов управления. Основные сценарии работы должны выполняться интуитивно и без лишних шагов.

Приложение должно включать следующие основные экраны:

- главный экран с выбором предмета и типа теста;
- экран прохождения теста с таймером, навигацией по заданиям и возможностью возврата к предыдущим вопросам;
- экран результатов с перечнем правильных и неправильных ответов, общей оценкой и временем выполнения;
- раздел статистики с историей решений, аналитикой по темам и динамикой успеваемости;
- интерфейс уведомлений о новых тестах, результатах и рекомендациях.

Веб-инструмент для преподавателей реализуется на базе Django Templates и должен обеспечивать удобный доступ к функциям создания и редактирования тестов, назначения их ученикам и просмотра статистики. Он обязан корректно работать в браузерах Google Chrome, Mozilla Firefox и Microsoft Edge. Минимальное разрешение экрана — 1366×768 пикселей.

Функциональные элементы веб-интерфейса включают:

- раздел «Создание теста» с возможностью добавления заданий из базы, создания новых вопросов и их упорядочивания;
- раздел «Мои тесты» для редактирования и публикации тестов;

2.4.2 Системные интерфейсы

Взаимодействие между клиентскими приложениями и серверной частью должно осуществляться через REST API, реализованный на ASP.NET Core. Для обмена данными используется формат JSON, транспортным протоколом

является HTTPS. Документация интерфейсов должна формироваться автоматически с использованием Swagger/OpenAPI.

Основные группы методов API включают:

- аутентификацию и авторизацию пользователей;
- доступ к базе тестов прошлых лет с возможностью фильтрации по предмету и году;
- сохранение и получение результатов прохождения тестов;
- предоставление статистики и аналитики ученику и преподавателю;
- управление пользовательскими тестами;
- получение и обработку уведомлений.

Для всех операций серверная часть обязана возвращать корректные коды HTTP-ответов. Ошибки должны сопровождаться информативными сообщениями в формате JSON. Авторизация осуществляется с помощью JWT-токенов, которые передаются в заголовках запросов. Доступ к данным разграничивается в зависимости от роли пользователя.

2.5 Требования к безопасности

Программный комплекс должен обеспечивать высокий уровень защиты персональных данных пользователей, устойчивость к внешним угрозам и предотвращение несанкционированного доступа. Безопасность рассматривается на уровне клиентских приложений, серверной части и базы данных.

Защита данных пользователей предусматривает:

- хранение паролей исключительно в виде хэш-сумм с использованием алгоритма bcrypt с солью, что исключает возможность их восстановления даже при компрометации базы;
- использование механизма политики сложности паролей, включающей минимальную длину и обязательное наличие различных типов символов;
- передачу всех данных только по защищённому протоколу HTTPS с использованием TLS версии 1.3, что предотвращает перехват информации;
- шифрование критически важных данных (например, электронных адресов) с использованием встроенных средств PostgreSQL;
- минимизацию хранения персональной информации: обязательными являются только имя, адрес электронной почты и роль пользователя, остальные данные носят факультативный характер.

Аутентификация и авторизация должны быть реализованы с использованием современных механизмов управления сессиями [8]. Для доступа к API применяются JWT-токены, генерируемые сервером ASP.NET Core и передаваемые в заголовках запросов. Каждый токен должен иметь ограниченный срок действия и автоматически обновляться по мере истечения. Система должна предусматривать возможность отзыва токена при подозрении на компрометацию. Разграничение прав доступа должно строиться на основе ролевой модели: «ученик» имеет доступ только к собственным данным и

назначенным тестам, «преподаватель» получает доступ к информации прикрепленных учеников, а доступ к административным функциям создания и модификации банка заданий возможен только через защищенный веб-интерфейс преподавателя.

Безопасность серверной части и базы данных должна обеспечиваться:

- использованием ORM Entity Framework Core для защиты от SQL-инъекций и безопасного взаимодействия с PostgreSQL;
- обязательной валидацией всех входных данных на стороне сервера;
- применением встроенных механизмов защиты ASP.NET Core и Django от XSS- и CSRF-атак [9];
- ограничением числа попыток входа для предотвращения атак перебора паролей [10];
- изоляцией базы данных от внешнего доступа, её подключение возможно только через сервер приложений;
- ведением журналов событий всех критически важных операций, включая входы в систему, создание и редактирование тестов, просмотр статистики и выдачу рекомендаций;
- регулярным обновлением зависимостей и применением актуальных патчей безопасности.

Мобильное приложение должно учитывать специфику использования на личных устройствах. Доступ к профилю должен быть защищен паролем или биометрической аутентификацией при наличии соответствующего оборудования. Хранение временных данных (например, незавершенных тестов в офлайн-режиме) должно осуществляться в защищенном локальном хранилище SQLite с шифрованием. После синхронизации данные должны удаляться из локального хранилища, чтобы минимизировать риск их компрометации при утере устройства.

Веб-инструмент для преподавателей должен защищаться дополнительными средствами. Доступ осуществляется только по имени пользователя и паролю с обязательной проверкой сложности. Сессии пользователей должны иметь ограниченное время жизни и завершаться при длительном бездействии. Должна быть реализована защита от подбора адресов URL и доступа к ресурсам без авторизации.

Важным элементом является обеспечение безопасности при администрировании и сопровождении системы. Необходимо предусмотреть многоуровневую систему мониторинга, включающую журналы событий, отслеживание подозрительных действий и автоматическое оповещение разработчиков при обнаружении аномалий. База данных должна резервироваться ежедневно, причём резервные копии должны храниться не менее семи дней и быть защищены от несанкционированного доступа. Все обновления системы должны проходить предварительное тестирование на предмет уязвимостей и не приводить к потере данных.

3 Проектирование программного комплекса

3.1 Архитектурные решения

Разрабатываемый программный комплекс проектируется на основе многослойной клиент–серверной архитектуры с чётким разделением ответственности между уровнями представления, прикладной логики и хранения данных. Такой подход обеспечивает модульность, масштабируемость и упрощает сопровождение системы.

На уровне представления функционируют два клиента: мобильное приложение для учеников и веб-инструмент для преподавателей. Мобильное приложение создаётся для операционной системы Android на языке Kotlin с использованием библиотек Jetpack Compose и Retrofit для работы с интерфейсом и сетевыми запросами. Оно обеспечивает выполнение всех функций, связанных с подготовкой учащегося: прохождение тестов, просмотр результатов, доступ к истории решений и получение рекомендаций. Веб-инструмент для преподавателей реализуется на основе Django Templates и предоставляет интерфейс для создания и редактирования тестов, назначения их прикреплённым ученикам. Интерфейс ориентирован на браузеры и мобильные устройства, что обеспечивает удобство работы с системой в разных сценариях.

Уровень прикладной логики представлен серверной частью, реализуемой на платформе ASP.NET Core 8 [3]. Сервер выполняет ключевые функции: аутентификацию и авторизацию пользователей, управление ролями, доступ к банку тестов прошлых лет, обработку пользовательских запросов, формирование статистики и рекомендаций, работу с уведомлениями. Для взаимодействия с клиентами используется REST API с передачей данных в формате JSON. Авторизация реализуется с применением JWT-токенов, которые передаются в заголовках HTTP-запросов. В серверной части используется модульная структура, включающая подсистемы управления пользователями, тестами, статистикой и аналитикой. Для устойчивости и удобства сопровождения сервер документируется с помощью Swagger [6].

Уровень хранения данных реализуется в СУБД PostgreSQL версии 15 [4]. В базе данных сохраняются сведения о пользователях, тестах, заданиях, результатах попыток, статистике и уведомлениях. Доступ к базе данных осуществляется исключительно через серверную часть с использованием ORM Entity Framework Core, что обеспечивает защиту от SQL-инъекций и унифицирует работу с данными. Для повышения отказоустойчивости предусматривается ежедневное резервное копирование базы, а в перспективе – настройка репликации для масштабируемости и балансировки нагрузки.

Архитектурные решения предусматривают использование только защищённого протокола передачи данных HTTPS. Развёртывание серверной части может осуществляться как на физических серверах, так и в облачных средах: Microsoft Azure или Amazon Web Services. Для удобства

развертывания и сопровождения должно быть предусмотрено использование контейнеризации на базе Docker. Такой подход обеспечит гибкость в управлении системой и снизит риски, связанные с зависимостью от конкретной инфраструктуры.

Физическая структура программного комплекса с указанием вычислительных узлов, их аппаратного и программного окружения отражена на диаграмме развёртывания, приведённой на рисунке 3.1.

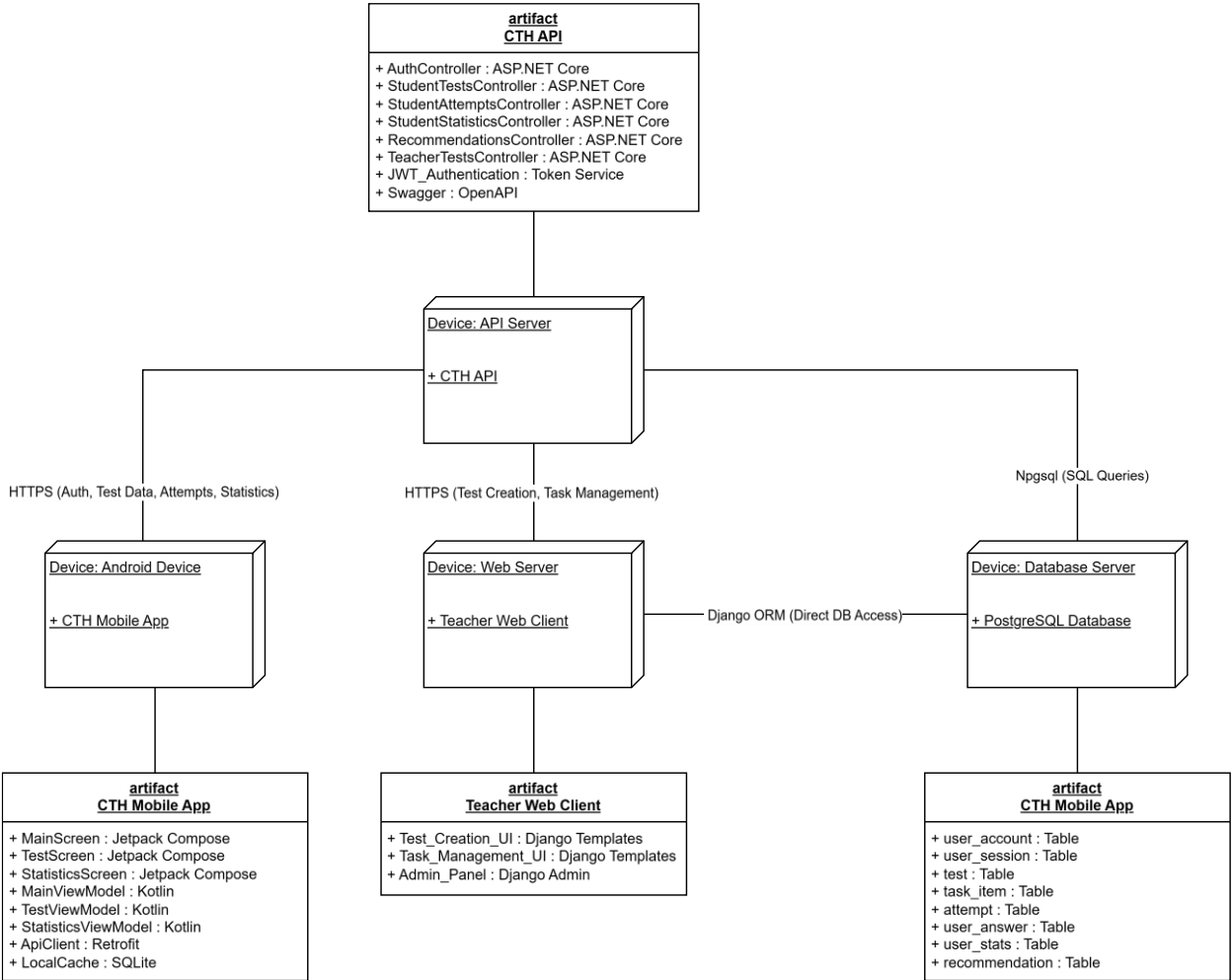


Рисунок 3.1 – Диаграмма развёртывания программного комплекса

Выбор многослойной архитектуры обусловлен необходимостью обеспечения масштабируемости и гибкости. В отличие от монолитного решения, разделение на уровни позволяет модернизировать отдельные компоненты без риска нарушения работы всего комплекса. Использование микросервисной архитектуры на данном этапе признано избыточным из-за ограниченного объёма функциональности и повышенной сложности сопровождения. Трёхуровневая архитектура является оптимальным вариантом для решения поставленных задач, так как сочетает относительную простоту реализации с возможностью дальнейшего расширения.

3.2 Модульная структура системы

Программный комплекс включает в себя ряд подсистем, каждая из которых отвечает за выполнение определённого набора функций. Модульная структура обеспечивает логическое разделение компонентов и их взаимодействие в рамках общей архитектуры.

Модуль аутентификации и авторизации обеспечивает регистрацию новых пользователей, вход в систему и контроль доступа. В данном модуле реализуются механизмы проверки учётных данных, генерации и валидации JWT-токенов, а также процедуры восстановления пароля через электронную почту [7]. Модуль поддерживает ролевую модель доступа и определяет права пользователей в зависимости от их статуса: ученик или преподаватель.

Модуль управления пользователями отвечает за хранение и обработку информации о зарегистрированных аккаунтах. Он обеспечивает создание, обновление и удаление учётных записей, управление ролями и связями между пользователями. В рамках этого модуля преподаватель имеет возможность добавлять учеников к своему аккаунту, просматривать их данные и при необходимости изменять статус доступа.

Модуль управления тестами реализует функциональность работы с тестовыми материалами. Он включает хранение заданий прошлых лет, поддержку поиска и фильтрации по предметам, темам и уровням сложности. В модуле предусмотрена возможность создания новых тестов преподавателями, редактирования существующих заданий, а также генерации смешанных тестов по параметрам, заданным пользователем.

Модуль прохождения тестов обеспечивает проведение тестирования учащихся в условиях, приближённых к экзаменационным. В него входят механизмы отображения заданий, отслеживания времени выполнения, сохранения промежуточных данных и фиксации финального результата. Результаты выполнения передаются в систему статистики и сохраняются в базе данных для последующего анализа.

Модуль статистики и аналитики отвечает за обработку накопленных данных о прохождении тестов. Он формирует отчёты по отдельным ученикам и группам, анализирует динамику успеваемости и распределение ошибок по темам. В рамках этого модуля реализуется подсистема автоматической генерации рекомендаций, которая определяет разделы программы, требующие повторения.

Модуль уведомлений используется для информирования пользователей о событиях в системе. Он формирует и отправляет сообщения о назначении новых тестов, появлении результатов, необходимости повторения тем и регулярной подготовке. Для мобильного приложения уведомления реализуются через push-сервисы, а веб-интерфейс использует встроенные механизмы оповещения.

Логическое разделение программного комплекса на

взаимодействующие модули и подсистемы с указанием их иерархии и информационных связей схематично показано на рисунке 3.2.

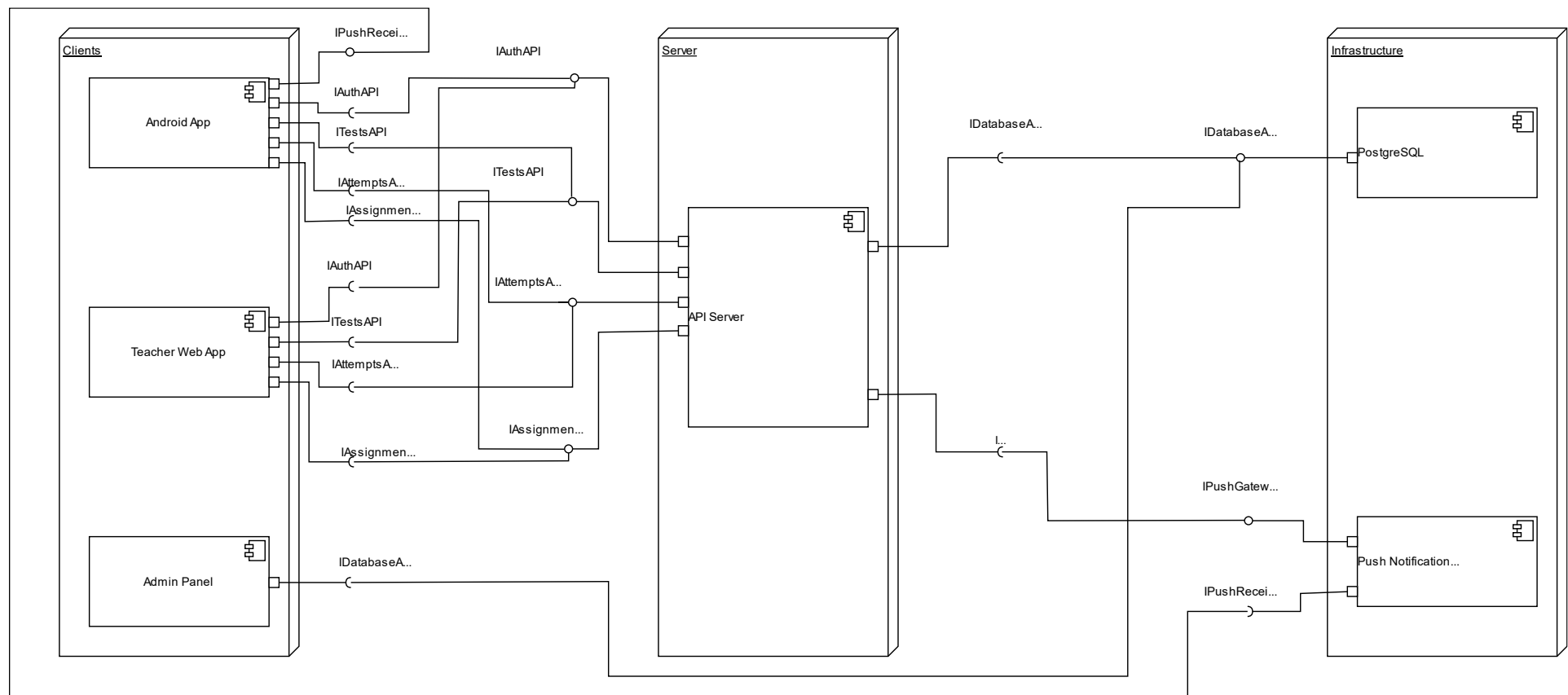


Рисунок 3.2 – Диаграмма модульной структуры программного комплекса

Модуль веб-интерфейса для преподавателей реализуется на базе Django Templates и предоставляет специализированный набор инструментов. Он включает функциональность создания и редактирования тестов, назначения их прикрепленным ученикам, просмотра их истории решений и анализа статистики. Интерфейс адаптирован для работы в браузере и позволяет выполнять все действия через удобные формы и таблицы.

Модуль административного управления данными используется для администрирования основных сущностей системы: пользователей, тестов, заданий. Он реализуется на основе встроенной панели Django Admin, что позволяет управлять содержимым базы данных через веб-интерфейс без необходимости создания отдельного интерфейса. Данный модуль используется разработчиками и администраторами системы, не предназначен для массового доступа преподавателей или учеников и выполняет функции технического сопровождения.

3.3 Взаимодействие компонентов

Взаимодействие компонентов программного комплекса строится по классической трёхуровневой схеме «клиент–сервер», предусматривающей чёткое разделение на уровень представления (клиентская часть), уровень бизнес-логики (серверная часть) и уровень хранения данных (база данных). Такая архитектура обеспечивает масштабируемость, независимость компонентов и упрощает сопровождение системы. Все коммуникации между клиентскими приложениями и серверной частью осуществляются по защищённому протоколу HTTPS с использованием REST API [5], что гарантирует конфиденциальность и целостность передаваемых данных. Обмен данными выполняется в текстовом формате JSON, который обеспечивает высокую скорость обработки и удобство интеграции между разнородными компонентами.

Мобильное приложение используется как учениками, так и преподавателями, при этом функциональные возможности интерфейса адаптируются в зависимости от роли пользователя. В первом случае приложение обеспечивает доступ к прохождению тестов в интерактивном режиме, автоматическому сохранению и последующему просмотру результатов, формированию детализированной статистики по каждому тесту, а также получению персонализированных рекомендаций на основе успеваемости. Во втором случае преподавательский интерфейс мобильного приложения дополняется расширенными возможностями: работа со статистикой прикрепленных учеников, просмотр динамики их успеваемости, анализ истории решений и выявление типичных ошибок. Все запросы, поступающие от мобильного клиента, передаются на сервер, где они проходят необходимую обработку, валидацию и сохраняются в базе данных.

Веб-инструмент для преподавателей реализован на основе Django Templates и выполняет ключевые функции по созданию и редактированию

тестов. Преподаватель формирует новый тест, выбирая задания из предварительно наполненного банка, либо создаёт собственные вопросы с настройкой параметров сложности, типов ответов и временных ограничений. После завершения редактирования преподаватель публикует тест и назначает его конкретному списку учеников или целым учебным группам. Все операции, выполняемые в веб-инструменте, осуществляются через REST API, что обеспечивает единый канал взаимодействия с серверной частью и согласованность данных. Следует отметить, что доступ к аналитике и детализированным результатам в веб-инструменте не предусмотрен – эти данные отображаются исключительно в мобильном приложении, что разграничивает сценарии использования и упрощает интерфейсы.

Серверная часть реализует всю бизнес-логику программного комплекса и отвечает за централизованную обработку всех запросов как от мобильного приложения, так и от веб-инструмента. В её функции входит: валидация входных данных, управление доступом на основе ролей (ученик, преподаватель, администратор), обработка результатов прохождения тестов с автоматическим подсчётом баллов, формирование статистических отчётов и рекомендаций. Для взаимодействия с базой данных используется ORM Entity Framework Core, который обеспечивает безопасный параметризованный доступ к информации, предотвращает SQL-инъекции и предоставляет централизованное хранение всех сущностей системы: пользователей, тестов, заданий, результатов и сессий.

Административная панель Django Admin взаимодействует с базой данных напрямую через встроенный ORM Django, минуя REST API. Она предназначена исключительно для администраторов системы и используется для управления основными сущностями: пользователями (регистрация, блокировка, назначение ролей), тестами (просмотр, модерация, удаление) и заданиями (добавление, редактирование, категоризация). В отличие от мобильного приложения и веб-инструмента, административная панель не подразумевает использования API и ориентирована на задачи технического обслуживания, сопровождения и оперативного исправления данных без участия конечных пользователей.

Временной порядок взаимодействия компонентов программного комплекса для различных сценариев удобно представить в виде диаграмм последовательности. На рисунке 3.3 изображён сценарий аутентификации и авторизации пользователя, включающий обмен JWT-токенами.

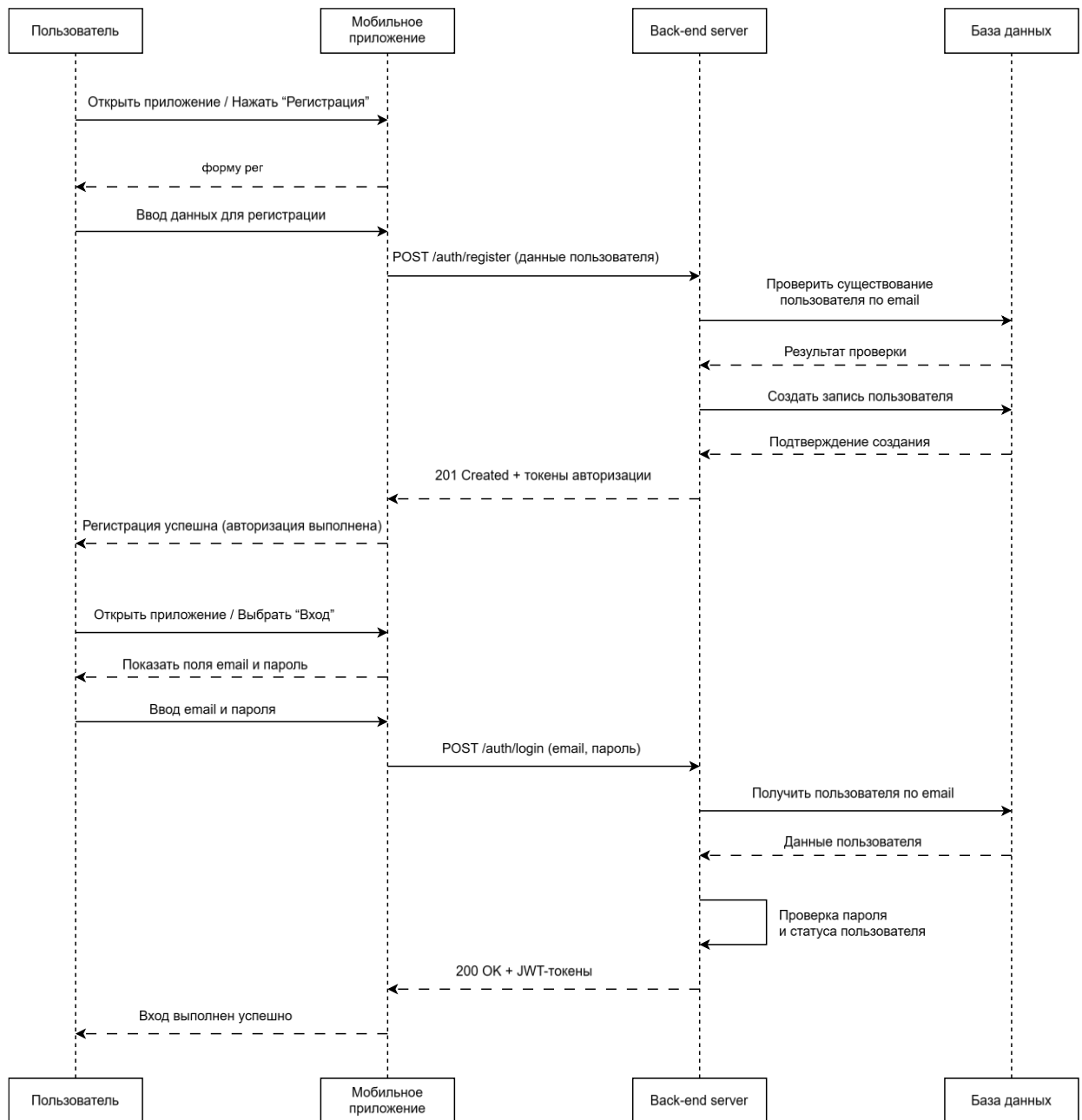


Рисунок 3.3 — Диаграмма последовательности для процесса аутентификации и авторизации пользователя

Сценарий прохождения теста, иллюстрирующий потоки данных между мобильным клиентом, сервером приложений и базой данных от момента выбора теста до получения окончательного результата, представлен на рисунке 3.4.

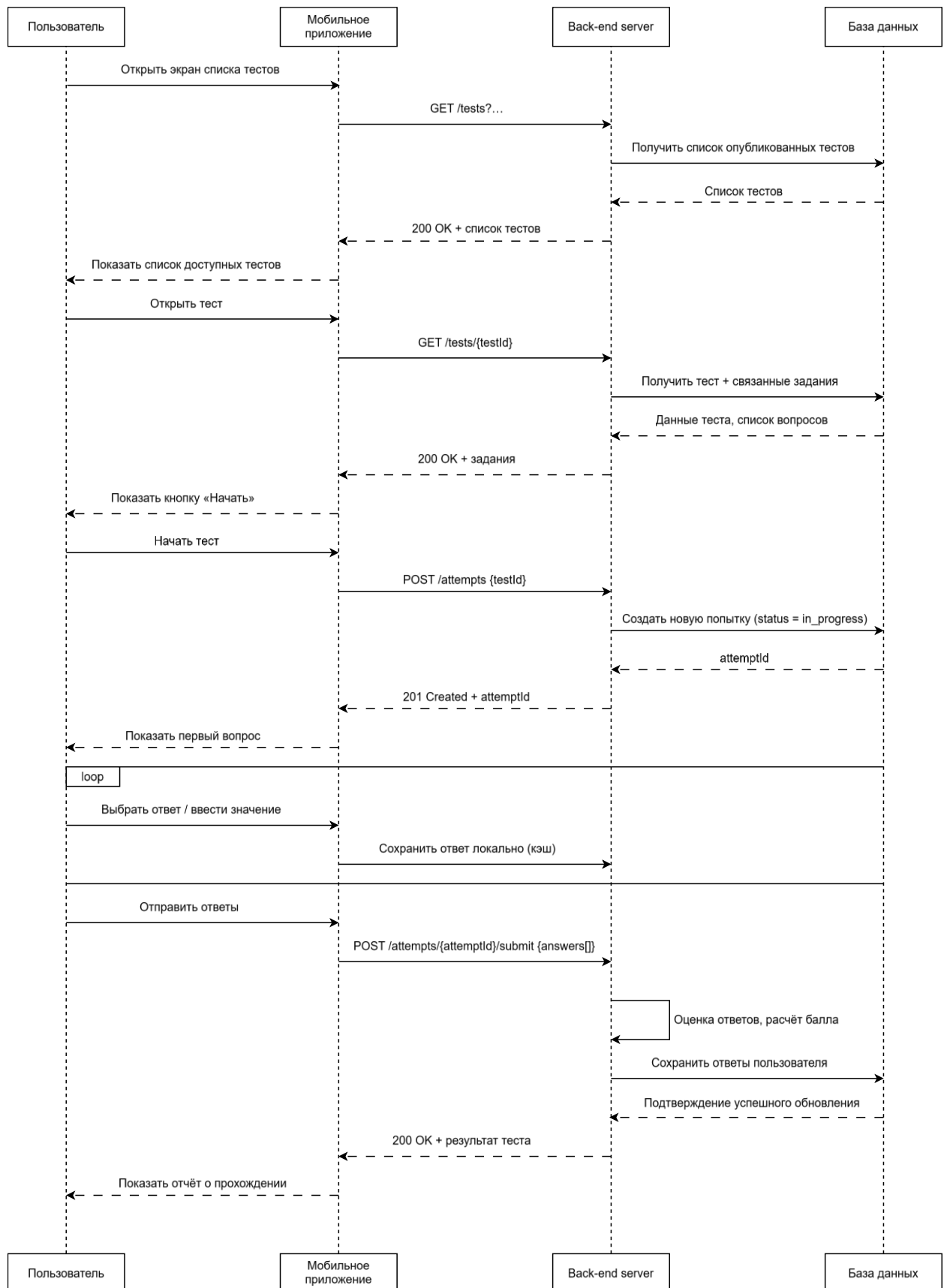


Рисунок 3.4 – Диаграмма последовательности для процесса прохождения теста

Таким образом, взаимодействие компонентов можно описать следующим образом:

- мобильное приложение ↔ серверная часть ↔ база данных;
- веб-инструмент (создание тестов) ↔ серверная часть ↔ база данных;
- административная панель Django Admin ↔ база данных напрямую через ORM.3.3 .

3.4 Сценарии использования системы

Ниже приведены ключевые сценарии взаимодействия пользователей и компонентов системы. Для каждого сценария указаны участники, предпосылки, последовательность действий и ожидаемые результаты, а также основные ветвления и пограничные ситуации:

1 Регистрация пользователя

Участники: гость.

Предпосылки: доступ к мобильному приложению Android.

Шаги:

1 Пользователь открывает экран регистрации и вводит следующие данные: имя, адрес электронной почты, пароль. Также осуществляется выбор роли из двух возможных вариантов: «ученик» или «преподаватель».

2 Клиентское приложение формирует и отправляет на сервер соответствующий запрос с использованием протокола HTTPS и формата JSON.

3 Сервер выполняет валидацию полученных данных, проверяя уникальность адреса электронной почты, соответствие пароля установленной политике сложности, а также корректность выбранной роли.

4 При успешном прохождении валидации в системе создаётся новая учётная запись, после чего сервер возвращает клиенту подтверждение регистрации и JWT-токен для обеспечения первичной сессии.

Результат: создана учётная запись, пользователь аутентифицирован.

Ветвления: e-mail уже зарегистрирован; слабый пароль; сетевой сбой – клиент показывает ошибку и предлагает повторить.

2 Вход и выход из системы

Участники: учитель, ученик.

Шаги:

1 Пользователь вводит в соответствующие поля адрес электронной почты и пароль для выполнения входа в систему.

2 Сервер проверяет корректность переданных учётных данных и в случае успеха выдаёт клиенту JWT-токен, имеющий ограниченный срок действия.

3 Клиентское приложение сохраняет полученный токен безопасным способом с использованием EncryptedSharedPreferences и Keystore, а затем применяет его при последующих запросах, добавляя в заголовки.

4 При выполнении выхода из системы клиент удаляет все локальные маркеры сессии, хранящиеся на устройстве.

Ветвления: неверные данные; превышен лимит попыток; истёкший токен – клиент запрашивает новый по повторному входу.

3 Привязка ученика к преподавателю

Участники: ученик, преподаватель.

Предпосылки: обе стороны зарегистрированы и вошли в систему.

Шаги:

1 Преподаватель в мобильном приложении создаёт ограниченный по времени код приглашения.

2 Ученик вводит код на экране «Привязка к преподавателю».

3 Сервер валидирует код, устанавливает связь «преподаватель – ученик».

4 Обеим сторонам отправляется подтверждение (push/внутреннее уведомление).

Ветвления: просроченный/неверный код; ученик уже привязан – система сообщает текущее состояние.

4 Просмотр доступных тестов

Участники: ученик, учитель.

Шаги:

1 Пользователь открывает раздел «Тесты».

2 Клиент запрашивает у сервера перечни: готовые тесты прошлых лет (с фильтрами «предмет», «год»), назначенные пользовательские тесты, конструктор смешанного теста.

3 Сервер возвращает списки с метаданными (длительность, число заданий, категории).

4 Результат: на экране список отфильтрованных тестов, каждая позиция в котором представлена названием теста и его метаданными.

5 Генерация смешанного теста

Участники: ученик.

Шаги:

1 Ученик выбирает предмет, задаёт количество заданий по категориям/темам и уровень сложности; при необходимости включает опцию «добавлять задания из тестов преподавателя».

2 Клиент отправляет параметры генерации на сервер.

3 Сервер валидирует доступность пула заданий под заданные параметры; фиксирует «состав» с уникальным seed (для воспроизводимости); возвращает сводку теста.

4 Ученик подтверждает запуск.

5 Ветвления: недостаточно заданий в одной из категорий – сервер возвращает допустимые пределы и предлагает скорректировать параметры.

Логика работы алгоритма генерации смешанного теста представлена на рисунке 3.9.

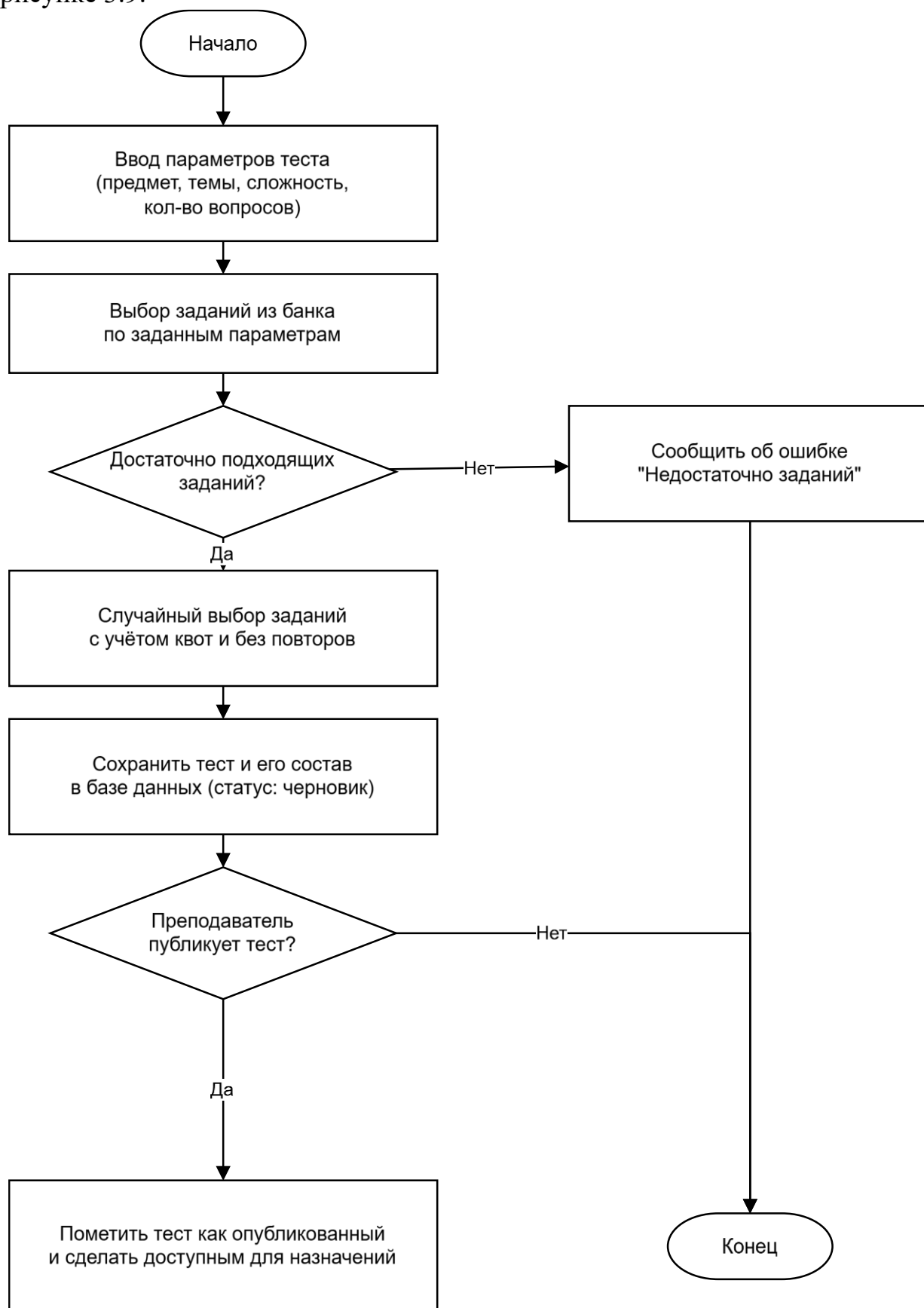


Рисунок 3.9 — Схема алгоритма генерации смешанного теста

6 Прохождение полного варианта теста прошлых лет

Участники: ученик.

Шаги:

1 Ученик выбирает вариант (предмет/год/номер), запускает тест; сервер фиксирует старт попытки.

2 Клиент отображает таймер, обеспечивает навигацию по вопросам, локально кэширует ответы.

3 При каждом ответе клиент сохраняет черновик локально.

4 По завершении ученик отправляет ответы на сервер.

Результат: сервер проверяет ответы, вычисляет результат, фиксирует попытку с временными метками и возвращает подробности.

7 Прохождение пользовательского теста (назначенного преподавателем)

Участники: ученик.

Шаги:

1 Ученик получает уведомление о назначенном тесте и видит его в разделе «Пользовательские».

2 Запускает тест; сервер фиксирует попытку, состав и параметры (таймер, порядок).

3 Дальнейшая логика идентична сценарию, описанному в пункте 6.

Ветвления: тест отозван преподавателем до запуска – скрыть из списка; попытка повторного запуска при политике «одна попытка» – отказ.

8 Продолжение прерванного теста

Участники: ученик.

Шаги:

1 При аварийном завершении/обрыве сети клиент хранит локальный снимок состояния: текущий вопрос, ответы, остаток времени.

2 При повторном входе клиент запрашивает у сервера статус попытки; если попытка активна – восстанавливает состояние; при строгом таймере – учитывается прошедшее время.

3 После восстановления сети локальные изменения синхронизируются.

Ветвления: истёк лимит времени – клиент переводит пользователя к экрану результатов.

9 Завершение теста и подсчёт результата

Участники: ученик, сервер.

Шаги:

1 Клиент отправляет финальный набор ответов с идентификатором попытки.

2 Сервер проверяет целостность запроса и выполняет оценку: первичные баллы, при наличии – шкалирование.

3 Сервер сохраняет попытку, помечает её завершённой, инициирует

пересчёт тематической статистики.

4 Клиент получает результат и отображает детальную раскладку.

Серверный алгоритм обработки завершения попытки схематично изображён на рисунке 3.10.

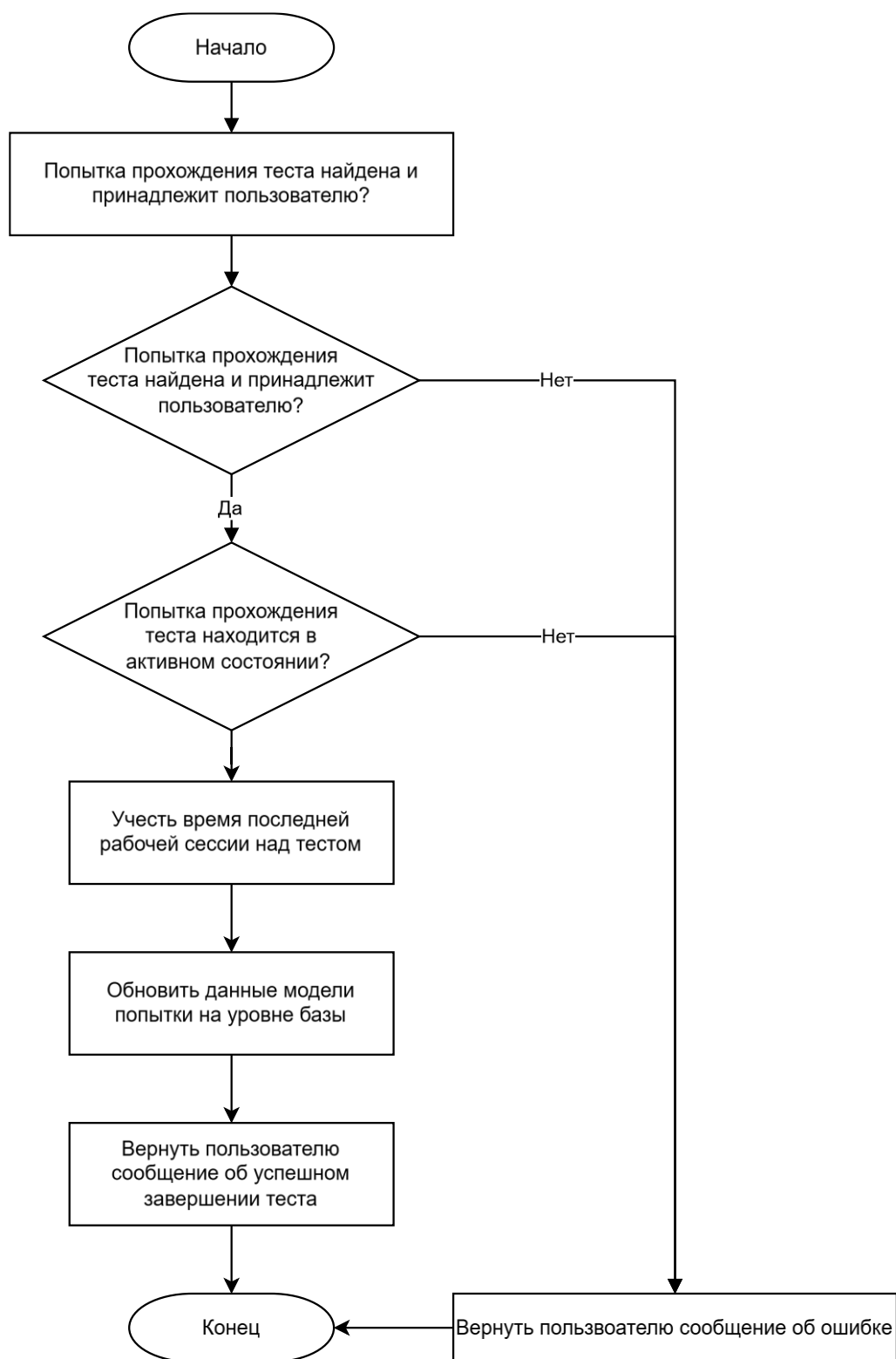


Рисунок 3.10 — Схема алгоритма завершения попытки тестирования на стороне API

10 Просмотр результатов и детальный разбор

Участники: ученик.

Шаги:

1 Ученик открывает экран результатов последней попытки или из истории.

2 Клиент запрашивает у сервера: общий балл, время выполнения, список заданий с пометками «верно/ошибка», привязку к темам/категориям и доступные пояснения.

3 Отображается послевыборочный анализ (ошибочные темы, доля верных ответов по категориям, среднее время ответа).

11 История решений и фильтрация

Участники: ученик.

Шаги:

1 Ученик открывает раздел «История».

2 Клиент отправляет фильтры: предмет, тип теста (прошлый год/смешанный/пользовательский), период дат.

3 Сервер возвращает список попыток с метаданными; при раскрытии записи – детали.

4 Результат: пользователю доступна полная трассируемость подготовки.

12 Получение и применение рекомендаций

Участники: ученик.

Шаги:

1 После завершения попытки сервер пересчитывает профиль тем: доля ошибок, давность, частота.

2 Формируется упорядоченный список «рекомендуемых к повторению» тем.

3 Клиент запрашивает рекомендации (или получает уведомление) и отображает блок с приоритетами; при переходе – предложение запустить тренировку по выбранным темам (смешанный мини-тест).

13 Уведомления

Участники: ученик/преподаватель.

Шаги:

1 Сервер регистрирует токен push-сервиса клиента.

2 При назначении теста, появлении результата или новых рекомендаций сервер отправляет уведомление.

3 Пользователь открывает уведомление и попадает на соответствующий экран (тест/результат/рекомендации).

Ветвления: отключённые уведомления – клиент показывает пункт «Новые события» внутри приложения и счётчик непрочитанных.

14 Создание пользовательского теста

Участники: преподаватель.

Шаги:

- 1 Преподаватель авторизуется в веб-инструменте.
- 2 Открывает «Создание теста», выбирает предмет и добавляет задания: из банка (поиск/фильтр) и/или новые вопросы (форма с полями: формулировка, варианты/ответ, категория, сложность, пояснение).
- 3 Сохраняет тест как черновик; сервер присваивает идентификатор и версию.
- 4 При необходимости редактирует порядок, ограничение времени, допустимое число попыток.
- 5 Публикует тест; статус меняется на «опубликован».

15 Назначение теста ученикам

Участники: преподаватель.

Шаги:

- 1 Преподаватель открывает в мобильном приложении страницу «Мои тесты», выбирает опубликованный тест, выбирает опцию назначить и из контекстного окна выбирает учеников, которым будет доступен этот тест.
- 2 Подтверждает назначение; сервер создаёт задания к исполнению для указанных учеников и отправляет уведомления.
- 3 В мобильном приложении учеников тест появляется в разделе «Пользовательские».

Ветвления: попытка назначить тест не своему ученику – отказ; повторное назначение при политике «одна попытка» – предупреждение.

Детальная логика операции назначения теста, реализованная в методе AssignTestToStudent сервиса AssignmentService, представлена на рисунке 3.11. Алгоритм включает последовательные проверки существования теста и наличия связи «преподаватель–ученик» с возвратом соответствующих кодов ошибок, после чего создаёт запись назначения с указанием deadline и количества попыток.

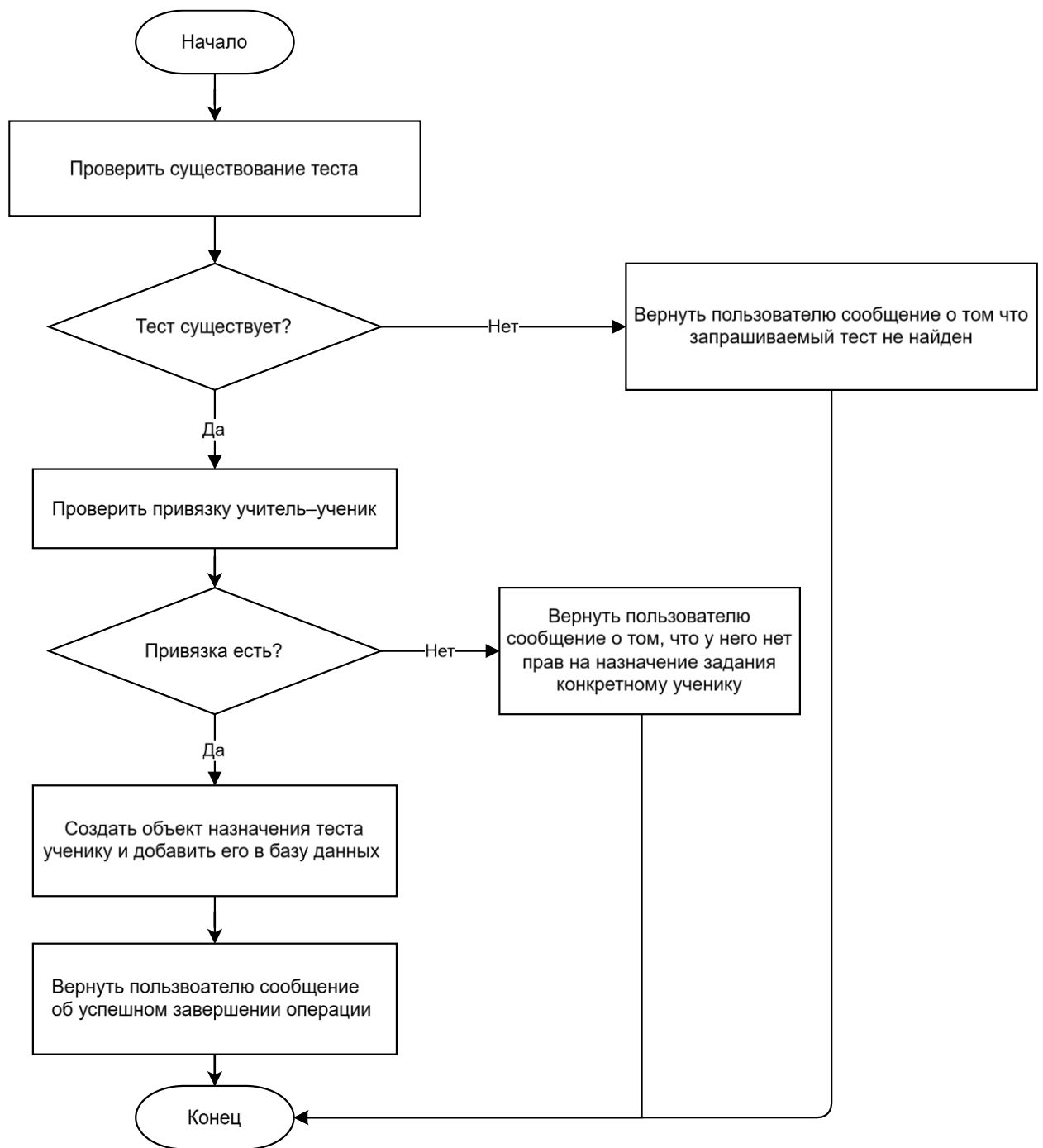


Рисунок 3.11 — Схема алгоритма назначения теста ученику

16 Просмотр результатов учеников преподавателем

Участники: преподаватель.

Шаги:

1 Преподаватель открывает раздел «Ученики».

2 Клиент запрашивает у сервера список прикрепленных учеников с основными метриками (последний результат, активность).

3 При выборе ученика запрашивается детальная статистика: история попыток, распределение ошибок по темам, динамика за период.

4 Преподаватель просматривает результаты по конкретному тесту и может рекомендовать темы к повторению (установка флага рекомендаций – опционально).

17 Управление банком заданий через Django Admin

Участники: администратор/разработчик.

Шаги:

1 Авторизация в Django Admin (отдельные доступы, вне пользовательского потока).

2 Просмотр и редактирование сущностей: задания, тесты прошлых лет, категории/темы, пользователи.

3 Сохранение изменений; ORM Django фиксирует записи в PostgreSQL.

Ветвления: конфликты уникальности; несоответствие схемы – система отклоняет строки с отчётом.

18 Обработка ошибок генерации/прохождения теста

Участники: ученик, сервер.

Случаи:

1 Недостаточно заданий под параметры смешанного теста – сервер возвращает допустимые пределы и список дефицитных категорий; клиент предлагает изменить параметры.

2 Повторная отправка результатов – сервер использует идемпотентность по идентификатору попытки и возвращает уже сохранённый результат.

3 Потеря сети при сдаче – клиент помещает результат в очередь синхронизации, показывает статус «Ожидание отправки», при восстановлении сети выполняет отправку автоматически.

19 Контроль доступа и защитные проверки

Участники: ученик/преподаватель, сервер.

Случаи:

1 Попытка доступа к чужим результатам – сервер возвращает 403 Forbidden.

2 Использование просроченного JWT – сервер возвращает 401 Unauthorized; клиент предлагает повторный вход.

3 Попытка назначения теста пользователю, не привязанному к преподавателю – отказ с объяснением причины.

20 Удаление привязки «преподаватель – ученик»

Участники: преподаватель или ученик.

Шаги:

- 1 Инициатор выбирает действие «Отвязать».
- 2 Сервер проверяет права, помечает связь неактивной; новые назначения невозможны.
- 3 История и результаты остаются в системе (для отчётности).

21 Сброс пароля

Участники: ученик/преподаватель.

Шаги:

- 1 Пользователь указывает e-mail на экране «Забыли пароль».
- 2 Сервер отправляет письмо со ссылкой на сброс; после перехода и задания нового пароля – инвалидирует предыдущие JWT.
- 3 Клиент выполняет новый вход.

22 Просмотр назначенных тестов и статуса выполнения

Участники: ученик.

Шаги:

- 1 Пользователь открывает раздел «Пользовательские».
- 2 Запрос статусов: «не начат», «в процессе», «завершён», дата назначения.
- 3 Переход к выполнению или просмотр результата, если завершён.

23 Отзыв/изменение опубликованного теста преподавателем

Участники: преподаватель.

Шаги:

- 1 Преподаватель в веб-инструменте открывает тест и меняет статус на «отозван».
- 2 Сервер прекращает выдачу теста в «Пользовательские» для не начавших; уже начатые попытки могут завершиться в соответствии с текущей политикой.

- 3 Ученики получают уведомление об изменении.

Для формализации функциональной декомпозиции программного комплекса на верхнем уровне применено моделирование в нотации IDEF0. На рисунке 3.5 приведена контекстная диаграмма IDEF0-A0 «Обеспечивать подготовку к централизованному тестированию». Она определяет основной бизнес-процесс системы, его входы (запросы пользователя), выходы (результаты и рекомендации), а также механизмы исполнения — мобильное приложение на Android/Kotlin, серверную часть на ASP.NET Core и базу данных PostgreSQL.

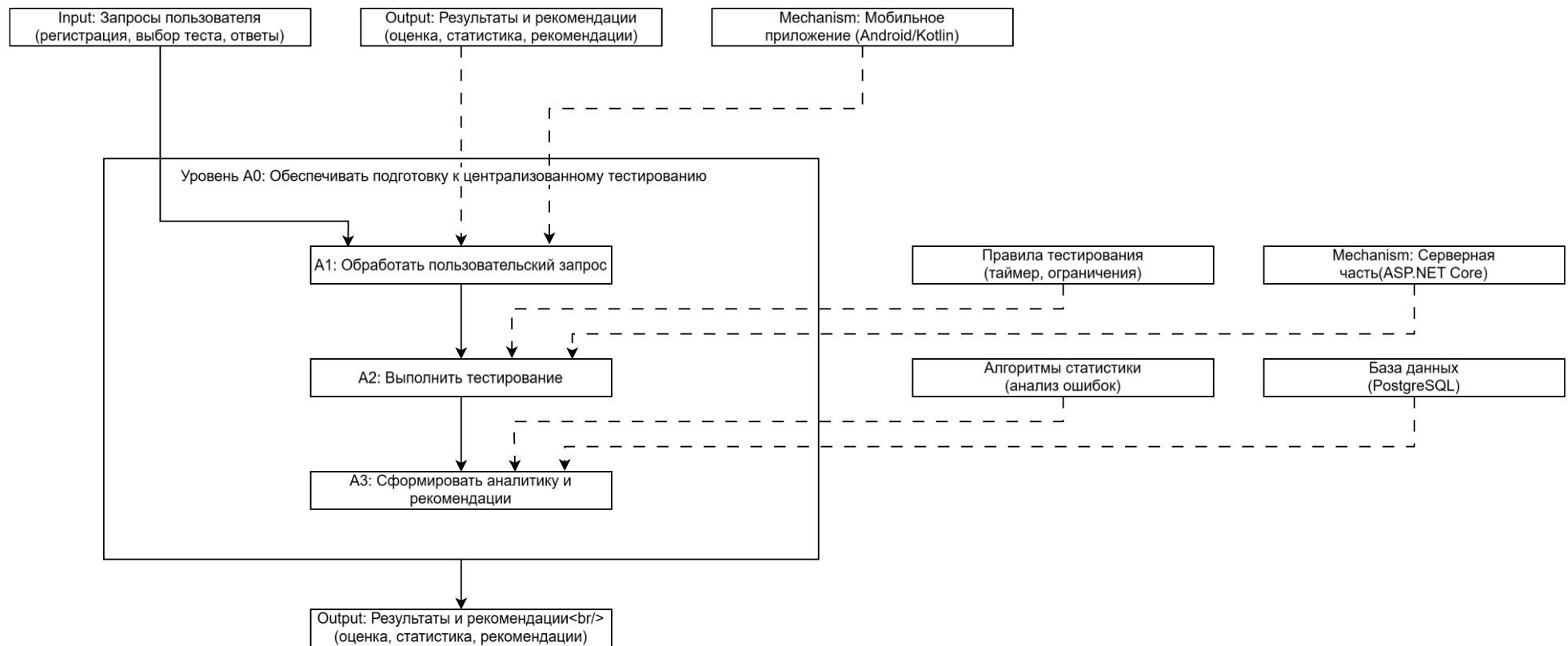


Рисунок 3.5 — Контекстная диаграмма IDEF0-A0 «Обеспечивать подготовку к централизованному тестированию»

Декомпозиция контекстного процесса на функциональные блоки первого уровня представлена на диаграммах IDEF0-A1, IDEF0-A2 и IDEF0-A3. На рисунке 3.6 изображена диаграмма IDEF0-A1 «Обработать пользовательский запрос», которая детализирует этапы аутентификации, получения списка тестов, валидации запроса и подготовки данных для последующего тестирования. В качестве механизмов задействованы контроллер аутентификации AuthController, HTTP-клиент ApiClient и репозиторий тестов TestRepository.

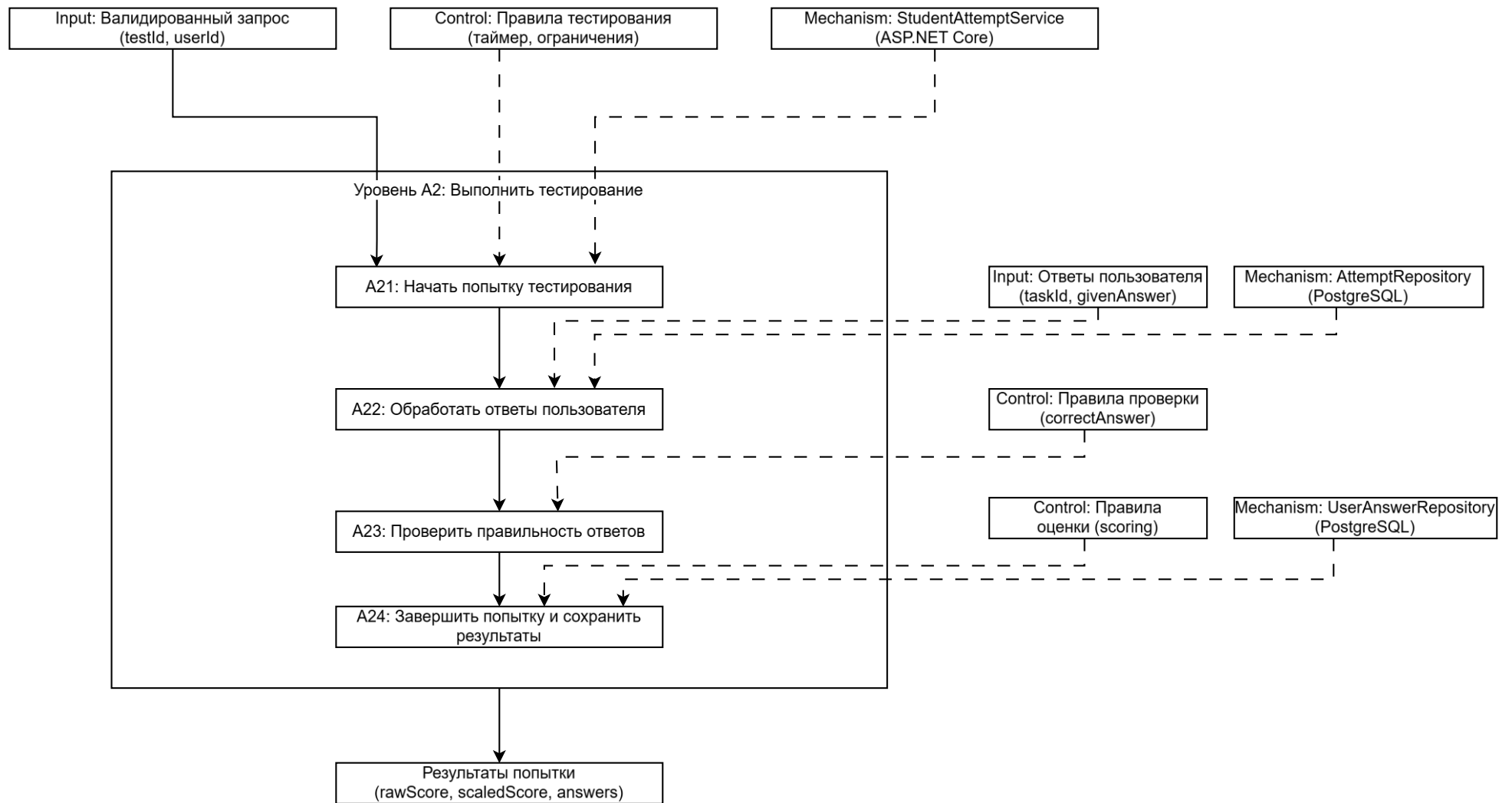


Рисунок 3.7 — Диаграмма IDEF0-A2 «Выполнить тестирование»

На рисунке 3.8 представлена диаграмма IDEF0-A3 «Сформировать аналитику и рекомендации». Данный блок описывает завершающий этап обработки результатов: расчёт статистики, анализ допущенных ошибок, формирование персонализированных рекомендаций по темам и подготовку итогового отчёта.

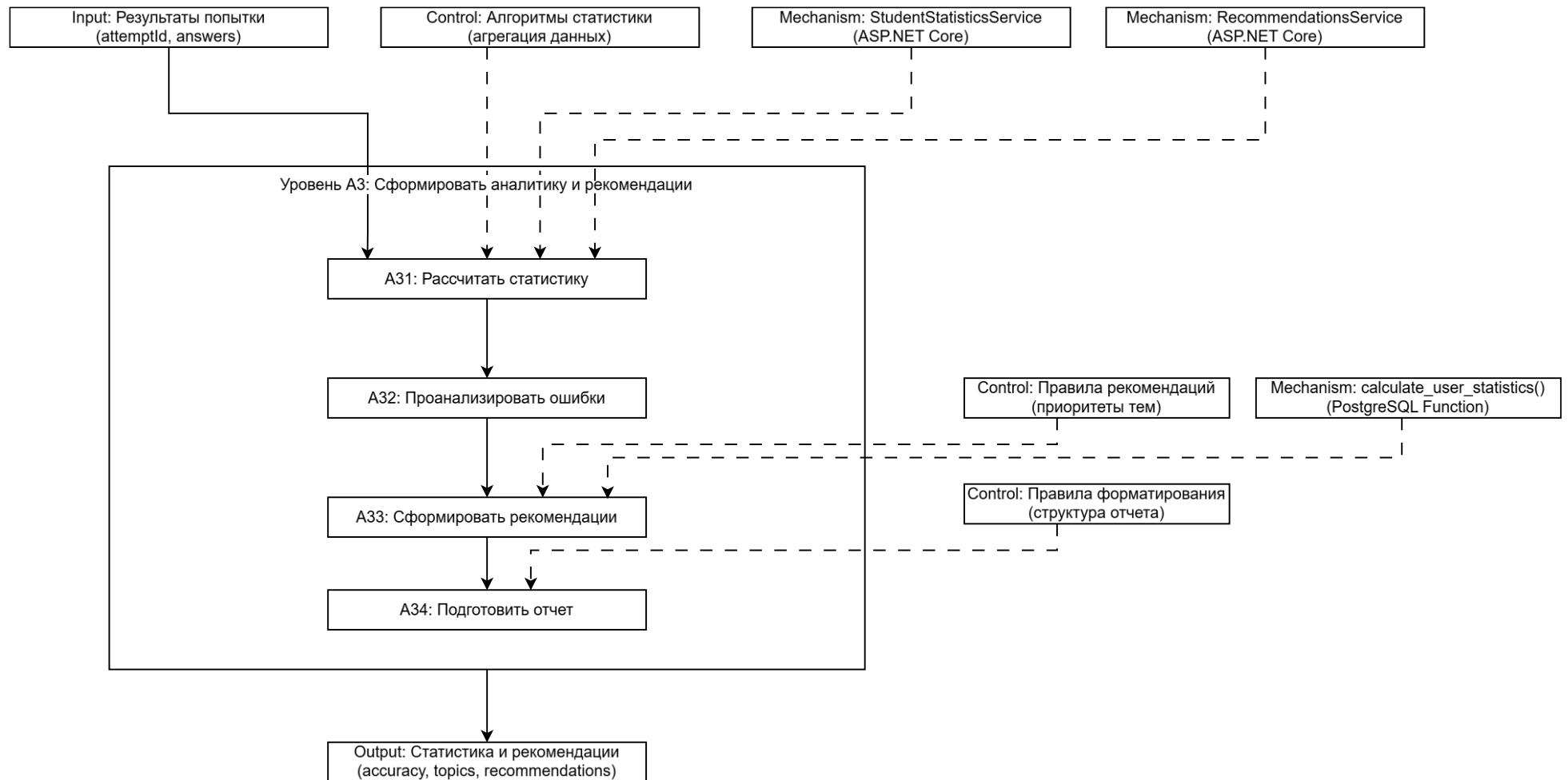


Рисунок 3.8 — Диаграмма IDEF0-A3 «Сформировать аналитику и рекомендации»

3.5 Обеспечение расширяемости и сопровождения

Обеспечивает возможность расширения функциональности без необходимости полной переработки существующих модулей.

В системе предусмотрена возможность добавления новых предметов и категорий заданий. База данных и бизнес-логика должны поддерживать расширение без изменения существующей структуры и сохранением совместимости с ранее введёнными данными.

Предусмотрена возможность добавления новых типов заданий (например, задания с кратким ответом, задания на сопоставление). Интерфейсы мобильного приложения и серверная часть должны поддерживать обработку различных форматов заданий без модификации базового механизма тестирования.

Модель пользователей поддерживает расширение перечня ролей. Добавление новых ролей не требует изменения основной логики аутентификации и авторизации.

Подсистема статистики и аналитики реализована таким образом, чтобы позволять добавлять новые отчёты и виды анализа без изменения существующих функций. Подсистема уведомлений должна поддерживать расширение перечня типов событий, при которых формируются уведомления.

Для сопровождения программного комплекса предусмотрена возможность регулярного обновления банка заданий и тестов через административную панель. Администратор имеет возможность редактировать, удалять и добавлять задания без вмешательства в код или прямого доступа к базе данных.

Предусмотрена возможность администрирования пользователей: изменение статуса, сброс пароля, блокировка доступа. Эти функции реализованы средствами административной панели.

Серверная часть фиксирует в журналах работы все ошибки, исключения и критические события. Логирование разработано централизованным и обеспечивает возможность анализа при эксплуатации системы.

Реализована возможность регулярного резервного копирования базы данных и возможность восстановления данных при сбое.

4 Проектирование и разработка базы данных

4.1 Обоснование выбора СУБД

Для хранения данных программного комплекса используется реляционная система управления базами данных PostgreSQL.

Выбор PostgreSQL обусловлен следующими факторами:

- система поддерживает транзакционную модель с соблюдением свойств ACID, что обеспечивает целостность и надёжность данных при одновременной работе большого числа пользователей;
- СУБД является свободно распространяемой и кроссплатформенной, что исключает необходимость приобретения лицензий и упрощает развёртывание на различных типах серверов;
- в системе поддерживается работа со сложными структурами данных (JSONB, массивы, полнотекстовый поиск), что позволяет хранить не только строго структурированные записи, но и дополнительные метаданные тестов и заданий [11];
- PostgreSQL обладает развитым инструментарием для проектирования и оптимизации запросов, включая поддержку индексов различных типов, оптимизатор запросов и механизмы репликации [12];
- СУБД имеет встроенные средства резервного копирования и восстановления, а также поддерживает масштабирование с помощью репликации и шардинга, что соответствует требованиям к отказоустойчивости и дальнейшему развитию системы.
- Для взаимодействия серверной части на ASP.NET Core с PostgreSQL используется библиотека Entity Framework Core. Это обеспечит унифицированный доступ к данным, поддержку миграций и независимость прикладного кода от конкретной реализации хранилища.
- Использование PostgreSQL позволяет организовать централизованное хранение информации о пользователях, ролях, тестах, заданиях, результатах прохождения и уведомлениях в единой базе данных с обеспечением безопасности и отказоустойчивости.

4.2 Логическая модель данных

Логическая модель данных программного комплекса обеспечивает хранение информации о пользователях, их ролях и связях, тестах и заданиях, попытках прохождения, результатах, статистике и уведомлениях.

Перед детальным проектированием логической структуры базы данных целесообразно рассмотреть потоки данных, циркулирующие между пользователями, процессами обработки и хранилищем. Диаграмма потоков данных верхнего уровня, на которой выделены внешние сущности, основные процессы преобразования информации и хранилище базы данных, приведена на рисунке 4.1.

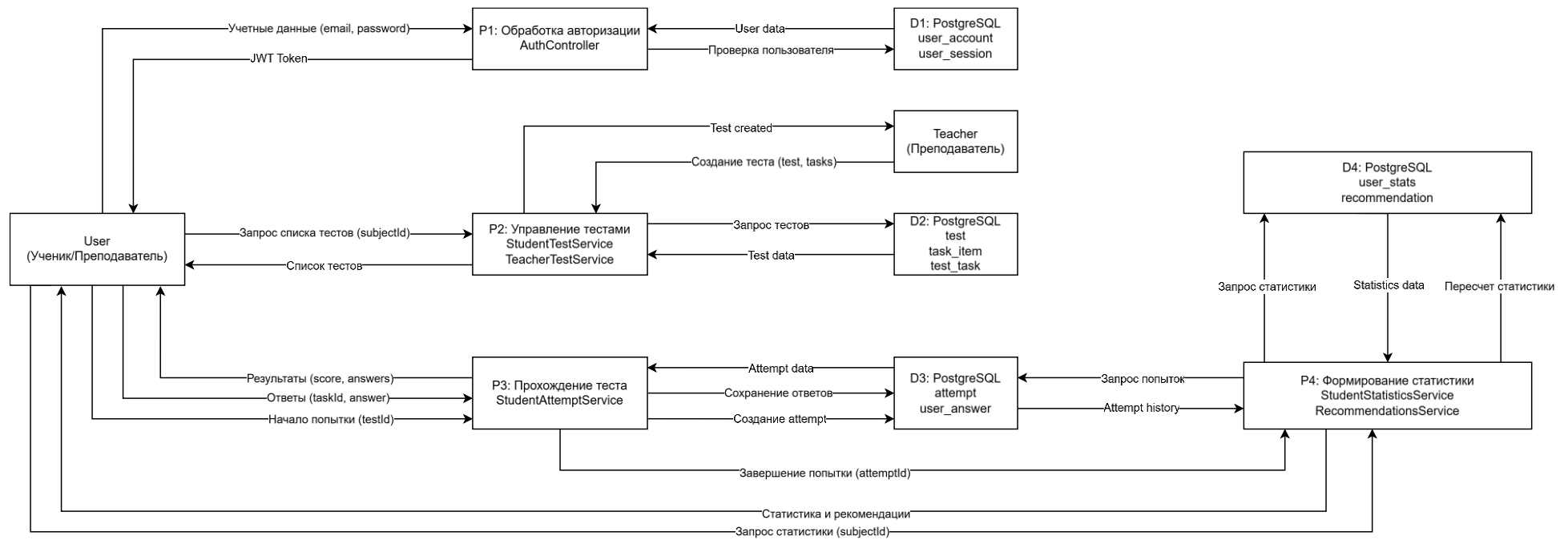


Рисунок 4.1 — Диаграмма потоков данных программного комплекса (DFD)

4.2.1 Сущность «Пользователь».

Сущность предназначена для хранения информации о зарегистрированных участниках системы.

Основные атрибуты:

уникальный идентификатор;

- имя;
- адрес электронной почты;
- пароль (хэшированное значение);
- дата регистрации;
- уникальный идентификатор роли;
- статус учётной записи.

Каждый пользователь связан с определённой ролью, определяющей права доступа. Пользователь может являться как учеником, так и преподавателем.

4.2.2 Сущность «Роль».

Сущность определяет уровень доступа пользователя.

Основные атрибуты:

- идентификатор роли;
- наименование роли.

Каждый пользователь связан ровно с одной ролью, но модель предусматривает возможность добавления новых ролей без изменения существующей структуры.

4.2.3 Сущность «Связь преподаватель—ученик»

Сущность хранит информацию о прикреплённых учениках для конкретных преподавателей.

Основные атрибуты:

- идентификатор связи;
- идентификатор преподавателя;
- идентификатор ученика;
- дата установления связи;
- статус.

Связь позволяет преподавателю назначать тесты своим ученикам и просматривать их статистику.

4.2.4 Сущность «Тест»

Сущность представляет собой набор заданий, сгруппированных для прохождения.

Основные атрибуты:

- уникальный идентификатор,
- идентификатор теста,
- название;
- тип теста;

- предмет;
- дата создания;
- автор;
- время выполнения;
- количество попыток.

4.2.5 Сущность «Задание»

Сущность хранит информацию о единичном задании.

Основные атрибуты:

- идентификатор;
- идентификатор теста;
- формулировка;
- тип задания;
- уровень сложности;
- тема;
- правильный ответ;
- пояснение;
- источник.

Задания могут использоваться в нескольких тестах.

4.2.6 Сущность «Состав теста»

Сущность связывает тест и задания, определяя их порядок и принадлежность.

Основные атрибуты:

- идентификатор;
- идентификатор теста;
- идентификатор задания;
- позиция в тесте.

Это позволяет одному заданию входить сразу в несколько тестов.

4.2.7 Сущность «Попытка»

Сущность фиксирует факт прохождения теста конкретным пользователем.

Основные атрибуты:

- идентификатор попытки;
- идентификатор пользователя;
- идентификатор теста;
- дата и время начала;
- дата и время завершения;
- статус;
- итоговый балл.

Каждая попытка связана с множеством ответов пользователя.

4.2.8 Сущность «Ответ пользователя»

Сущность хранит данные о решении конкретного задания в рамках попытки.

Основные атрибуты:

- идентификатор ответа;
- идентификатор попытки;
- идентификатор задания;
- введенный ответ;
- отметка о правильности;
- затраченное время.

Эта сущность позволяет восстанавливать полную историю прохождения теста.

4.2.9 Сущность «Уведомление»

Сущность отвечает за хранение уведомлений, отправленных пользователю.

Основные атрибуты:

- идентификатор уведомления;
- идентификатор пользователя;
- тип уведомления;
- текст сообщения;
- дата создания;
- статус.

4.2.10 Сущность «Рекомендация»

Сущность содержит сведения о темах, рекомендуемых пользователю для повторения.

Основные атрибуты:

- идентификатор рекомендации;
- идентификатор пользователя;
- идентификатор предмета;
- тема;
- основание рекомендации.

Формируется на основе статистики и попыток.

4.2.11 Сущность «Предмет»

Назначение: нормализованное представление предметной области экзамена.

Основные атрибуты:

- идентификатор предмета;
- название предмета.

Совокупность описанных сущностей, их атрибутов и связей в графическом виде представлена на ER-диаграмме, отражающей логическую модель данных программного комплекса, на рисунке 4.2.

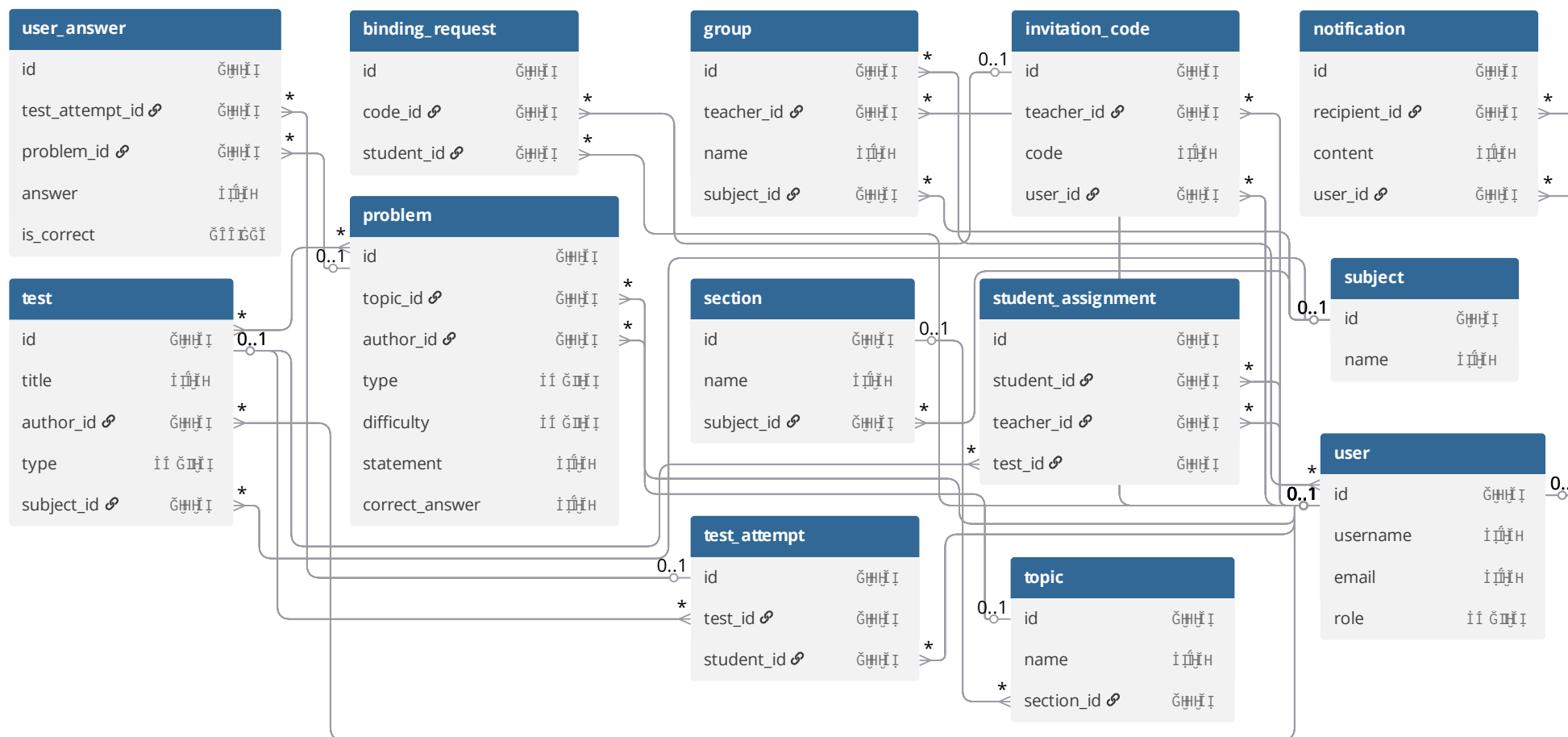


Рисунок 4.2 — Entity–relationship диаграмма программного средства

Связи между сущностями:

- один пользователь связан с одной ролью;
- один преподаватель может быть связан с множеством учеников, один ученик может быть связан с несколькими преподавателями (многие-ко-многим через сущность «Связь преподаватель—ученик»);
- один тест связан с множеством заданий (через «Состав теста»), одно задание может входить в несколько тестов;
- один пользователь может иметь множество попыток, каждая попытка относится к одному тесту;
- каждая попытка включает множество ответов пользователя;
- одному пользователю может быть отправлено множество уведомлений;
- каждому пользователю может быть назначено множество рекомендаций;

4.3 Физическая модель данных

Физическая модель определяет состав таблиц, типы данных, ключи и ограничения, а также индексацию и правила ссылочной целостности в СУБД PostgreSQL. Ниже приведены требования к структуре базы. Формулировки даны в нормативной форме без SQL-кода, но с указанием рекомендуемых типов PostgreSQL.

4.3.1 Общие требования и соглашения

- 1 Идентификаторы записей имеют тип `uuid` (PK), генерируемый на сервере приложений.
- 2 Для временных меток используется `timestampz` (UTC). Поля аудита: `created_at timestampz NOT NULL`, `updated_at timestampz NOT NULL`.
- 3 Текстовые поля: короткие справочные — `varchar(n)` с разумной длиной, длинные — `text`.
- 4 Логические флаги — `boolean NOT NULL` с значениями по умолчанию там, где необходимо.
- 5 Значимые перечисления (вид теста, тип уведомления и т. п.) хранятся как `smallint` согласно подходу `code first`.
- 6 Внешние ключи (FK) — `uuid` с явными политиками `ON DELETE/UPDATE`. Для справочников — `RESTRICT`, для зависимых записей — `CASCADE/SET NULL` по правилам ниже.
- 7 На внешние ключи и поля, используемые в фильтрации, созданы B-Tree индексы.

4.3.2 Структура моделей базы данных

1 Таблица `role`

Назначение: роли доступа.

Колонки:

- role_id uuid PK
- role_name varchar(32) UNIQUE NOT NULL
- created_at, updated_at

Ограничения: уникальность role_name.

2 Таблица user_account

Назначение: пользователи системы.

Колонки:

- user_id uuid PK
- email varchar(256) UNIQUE NOT NULL
- password_hash varchar(255) NOT NULL
- name varchar(128) NOT NULL
- status varchar(16) NOT NULL (active|blocked|pending) с CHECK
- role_id uuid NOT NULL → role(role_id)
- registered_at timestamptz NOT NULL
- created_at, updated_at

Ограничения/политики:

FK role_id ON UPDATE RESTRICT, ON DELETE RESTRICT.

Уникальность email.

3 Таблица subject

Назначение: предметы.

Колонки:

- subject_id uuid PK
- subject_code varchar(32) UNIQUE NOT NULL
- subject_name varchar(128) UNIQUE NOT NULL
- is_active boolean NOT NULL DEFAULT true
- created_at, updated_at

4 Таблица topic

Назначение: темы/категории внутри предмета (возможна иерархия).

Колонки:

- topic_id uuid PK
- subject_id uuid NOT NULL → subject(subject_id)
- topic_name varchar(256) NOT NULL
- topic_code varchar(64) (опционально, UNIQUE в рамках предмета)
- topic_parent_id uuid NULL → topic(topic_id) (для иерархий)
- is_active boolean NOT NULL DEFAULT true
- created_at, updated_at

Ограничения:

Уникальность пары (subject_id, topic_name).

FK topic_parent_id ON DELETE SET NULL.

5 Таблица exam_source (опционально)

Назначение: нормализация источников официальных вариантов.

Колонки:

- exam_source_id uuid PK
- year integer NOT NULL CHECK (year BETWEEN 2000 AND 2100)

- variant_number integer NULL
- issuer varchar(128) (например, организатор)
- notes text
- created_at, updated_at

Ограничения:

Уникальность комбинации (year,variant_number) при variant_number NOT NULL.

6 Таблица task

– назначение: задания.

Колонки:

- task_id uuid PK
- subject_id uuid NOT NULL → subject(subject_id)
- topic_id uuid NULL → topic(topic_id)
- exam_source_id uuid NULL → exam_source(exam_source_id)
- task_type varchar(24) NOT NULL (single|multiple|numeric|match|text ...)

с CHECK

- difficulty smallint NOT NULL CHECK (difficulty BETWEEN 1 AND 5)
- statement text NOT NULL
- correct_answer jsonb NOT NULL (структура зависит от task_type)
- explanation text NULL
- is_active boolean NOT NULL DEFAULT true
- created_at, updated_at

Политики:

- FK topic_id ON DELETE SET NULL.
- FK exam_source_id ON DELETE SET NULL.

7 Таблица test

Назначение: тестовые наборы.

Колонки:

- test_id uuid PK
- subject_id uuid NOT NULL → subject(subject_id)
- test_kind varchar(16) NOT NULL (PAST_EXAM|CUSTOM|MIXED) с

CHECK

- title varchar(256) NOT NULL
- author_id uuid NULL → user_account(user_id) (для CUSTOM)
- time_limit_sec integer NULL CHECK (time_limit_sec > 0)
- attempts_allowed smallint NULL CHECK (attempts_allowed > 0)
- is_published boolean NOT NULL DEFAULT false
- created_at, updated_at

Ограничения:

Если test_kind='CUSTOM' → author_id NOT NULL (проверка реализуется на уровне бизнес-логики или триггером).

8 Таблица test_task

Назначение: состав теста.

Колонки:

- test_task_id uuid PK
- test_id uuid NOT NULL → test(test_id)
- task_id uuid NOT NULL → task(task_id)
- position integer NOT NULL CHECK (position > 0)
- score_weight numeric(5,2) NULL
- created_at, updated_at

Ограничения:

- уникальность пары (test_id,task_id).
- уникальность (test_id,position).

9 Таблица teacher_student

Назначение: прикрепления «преподаватель—ученик».

Колонки:

- teacher_student_id uuid PK
- teacher_id uuid NOT NULL → user_account(user_id)
- student_id uuid NOT NULL → user_account(user_id)
- status varchar(16) NOT NULL (active|revoked) с CHECK
- created_at, updated_at

Ограничения:

- уникальность пары (teacher_id,student_id) при status='active'.

Политики:

- FK на user_account с ON DELETE CASCADE (удаление аккаунта снимает привязки).

10 Таблица assignment

Назначение: факт назначения теста ученику.

Колонки:

- assignment_id uuid PK
- test_id uuid NOT NULL → test(test_id)
- teacher_id uuid NOT NULL → user_account(user_id)
- student_id uuid NOT NULL → user_account(user_id)
- assigned_at timestampz NOT NULL
- due_at timestampz NULL
- status varchar(16) NOT NULL (active|revoked|expired) с CHECK
- created_at, updated_at

Ограничения:

- индивидуальные политики попыток завязываются на assignment_id

Политики:

- FK на пользователей — ON DELETE CASCADE.

11 Таблица attempt

Назначение: попытки прохождения тестов.

Колонки:

- attempt_id uuid PK
- test_id uuid NOT NULL → test(test_id)
- user_id uuid NOT NULL → user_account(user_id)
- assignment_id uuid NULL → assignment(assignment_id) (если запуск из

назначения)

- started_at timestamptz NOT NULL
- finished_at timestamptz NULL
- status varchar(16) NOT NULL (in_progress|finished|timed_out|aborted) с

CHECK

- raw_score numeric(6,2) NULL
- scaled_score numeric(6,2) NULL
- duration_sec integer NULL
- seed bigint NULL (для воспроизводимости состава в смешанных

тестах)

- created_at, updated_at

Ограничения:

Если assignment_id указан — пара (user_id, assignment_id) должна удовлетворять политике количества попыток (на уровне бизнес-логики/триггера).

Политики:

– FK assignment_id ON DELETE SET NULL (история попыток сохраняется).

12 Таблица user_answer

Назначение: ответы по заданиям внутри попытки.

Колонки:

- answer_id uuid PK
- attempt_id uuid NOT NULL → attempt(attempt_id)
- task_id uuid NOT NULL → task(task_id)
- given_answer jsonb NOT NULL
- is_correct boolean NOT NULL
- time_spent_sec integer NULL
- created_at, updated_at

Ограничения:

– уникальность пары (attempt_id, task_id) — одно задание один ответ на попытку.

Политики:

- ON DELETE CASCADE по attempt_id.

13 Таблица recommendation

Назначение: рекомендации к повторению.

Колонки:

- recommendation_id uuid PK
- user_id uuid NOT NULL → user_account(user_id)
- subject_id uuid NOT NULL → subject(subject_id)
- topic_id uuid NOT NULL → topic(topic_id)
- priority smallint NOT NULL CHECK (priority BETWEEN 1 AND 5)
- reason_code varchar(32) NOT NULL (правило генерации)
- created_at, updated_at

Ограничения:

– уникальность (user_id,topic_id) в активных рекомендациях.

14 Таблица notification

Назначение: уведомления.

Колонки:

– notification_id uuid PK

– user_id uuid NOT NULL → user_account(user_id)

– type varchar(24) NOT NULL

(assignment|result|recommendation|reminder) с CHECK

– payload jsonb NULL (метаданные: test_id, assignment_id и т. п.)

– status varchar(16) NOT NULL (unread|read) с CHECK

– created_at, updated_at

Индексы: по user_id, status, created_at для сортировки ленты.

4.3.3 Правила ссылочной целостности

Удаление пользователя: CASCADE для teacher_student, assignment, attempt, notification; RESTRICT для записей-справочников или перевод в «анонимизировано» по политике безопасности (вариант решается в проекте).

Удаление теста: CASCADE для test_task, assignment; RESTRICT для существующих attempt (историю не удалять). При необходимости — флаг is_published=false и логическое удаление.

Удаление темы/предмета: RESTRICT, либо перевод связанных сущностей в NULL/is_active=false (по согласованной политике).

Удаление попытки: CASCADE удаляет user_answer.

4.3.4 Индексация

Уникальные индексы: user_account(email), role(role_name), subject(subject_code), subject(subject_name), topic(subject_id, topic_name), test_task(test_id, task_id), test_task(test_id, position).

FK-индексы: user_account(role_id), topic(subject_id), task(subject_id), task(topic_id), task(exam_source_id), test(subject_id), test(author_id), test_task(test_id), test_task(task_id), teacher_student(teacher_id), teacher_student(student_id), assignment(test_id), assignment(teacher_id), assignment(student_id), attempt(test_id), attempt(user_id), attempt(assignment_id), user_answer(attempt_id), user_answer(task_id), user_stats(user_id), user_stats(subject_id), user_stats(topic_id), recommendation(user_id), recommendation(topic_id), notification(user_id).

Запросы по ленте: индекс notification(user_id, status, created_at DESC).

История попыток: индекс attempt(user_id, started_at DESC).

Фильтрация тестов прошлых лет: индекс task(exam_source_id), test(test_kind, subject_id).

5 Программная реализация

5.1 Выбор средств и инструментов разработки

Для реализации серверной части программного комплекса использована платформа ASP.NET Core. Она обеспечивает создание REST API, модульную архитектуру и высокую производительность при работе с запросами. Для доступа к базе данных должна применяться технология Entity Framework Core как объектно-реляционный маппер, позволяющий выполнять миграции схемы и работать с данными на уровне объектов.

Для разработки административного веб-инструмента преподавателя используется среда Django с системой шаблонов Django Templates. Такой выбор обоснован наличием встроенной административной панели для обслуживания системы, а также развитым механизмом маршрутизации и форм. Это решение позволяет обеспечить веб-доступ к функциям формирования и редактирования тестов, а также управлению пользовательскими связями.

Клиентская часть для учеников и преподавателей реализована в виде мобильного приложения на языке Kotlin с использованием официальных библиотек Android. Для построения интерфейсов используется Jetpack Compose, обеспечивающий декларативный подход и модульность. Дополнительно могут применяться Android Jetpack-компоненты для навигации, управления состоянием и работы с локальным хранилищем.

В качестве системы управления базами данных используется PostgreSQL. Эта СУБД является свободно распространяемой, поддерживает транзакции, индексацию и репликацию, что соответствует требованиям к надёжности и расширяемости [13].

При разработке используются современные системы контроля версий. Основной системой хранения исходного кода является Git, с размещением репозитория в сервисах GitHub. Для документирования API и взаимодействия компонентов используется спецификация Swagger, которая позволяет обеспечить единый формат описания интерфейсов и упростить разработку клиентских приложений.

Для сборки и управления зависимостями применяются стандартные средства: Gradle для мобильного приложения, NuGet для ASP.NET Core и pip/venv для Python-проектов. Это обеспечивает управляемость и воспроизводимость окружений.

Для тестирования используются встроенные средства каждой платформы: xUnit для .NET, pytest для Python и JUnit для Kotlin.

Для обеспечения удобства развертывания допускается использование контейнеризации с помощью Docker, что упростит настройку окружения и переносимость между серверами.

5.2 Структуре серверной части

Серверная часть реализована на платформе ASP.NET Core и обеспечивает выполнение бизнес-логики программного комплекса. Архитектура сервера многослойна, с выделением следующих уровней: уровень представления API, уровень прикладной логики и уровень доступа к данным.

На уровне представления API реализованы контроллеры для следующих групп функций:

- аутентификация и авторизация пользователей;
- управление пользователями и ролями;
- управление тестами прошлых лет, пользовательскими тестами и смешанными тестами;
- управление заданиями и их атрибутами;
- управление связями преподаватель—ученик и назначениями тестов;
- обработка попыток и ответов пользователей;
- предоставление статистики и аналитики;
- генерация и доставка уведомлений.

На уровне прикладной логики реализованы сервисы, обеспечивающие работу основных функциональных модулей:

- сервис пользователей (регистрация, вход, сброс пароля, изменение профиля);
- сервис тестов (создание, редактирование, удаление, генерация смешанных тестов);
- сервис заданий (управление содержимым банка заданий и их категориями);
- сервис попыток (фиксация старта, проверка ответов, подсчёт результатов, завершение попытки);
- сервис статистики (агрегация данных, формирование аналитики по темам и предметам);
- сервис рекомендаций (определение тем к повторению на основе статистики);
- сервис уведомлений (формирование событий, отправка push-сообщений, фиксация статуса уведомлений).

На уровне доступа к данным используется объектно-реляционное отображение (ORM) Entity Framework Core, которое обеспечивает абстракцию от конкретной реализации базы данных и позволяет оперировать объектами предметной области вместо прямых SQL-запросов. В рамках данного уровня определены все сущности, используемые в программном комплексе: пользователь, роль, предмет, тема, задание, тест, состав теста, назначение, попытка, ответ, статистика, рекомендация, уведомление. Каждая из перечисленных сущностей представлена соответствующей моделью, описывающей её структуру и атрибуты. Для каждой модели в явном виде определены конфигурации, включающие описание первичных и внешних

ключей, необходимые индексы для ускорения выполнения запросов, а также ограничения целостности данных, такие как уникальность полей, допустимые значения и обязательность заполнения.

Серверная часть программного комплекса поддерживает документирование интерфейсов прикладного программирования с помощью спецификации OpenAPI (реализованной посредством Swagger). Документация позволяет разработчикам и обслуживающему персоналу ознакомиться со всеми доступными конечными точками API. Для каждой конечной точки в документации указаны используемый HTTP-метод, маршрут (URL), набор входных параметров с их типами и обязательностью, формат выходных данных, а также перечень возможных кодов ответов с описанием соответствующих ситуаций (успешное выполнение, ошибки валидации, отсутствие прав доступа, внутренняя ошибка сервера и другие).

Взаимодействие с базой данных в системе выполняется исключительно через уровень доступа к данным, описанный выше. Прямое обращение к базе данных из контроллеров (обработчиков HTTP-запросов) или сервисов (содержащих бизнес-логику) не допускается архитектурой комплекса. Такой подход обеспечивает чёткое разделение ответственности между компонентами, упрощает тестирование и модификацию логики доступа к данным без изменения остальных частей системы.

Серверная часть также содержит подсистему логирования, предназначенную для регистрации всех критических событий, возникающих в процессе работы, ошибок выполнения и исключений [14]. Логирование выполняется с обязательной привязкой к идентификатору пользователя, инициировавшего действие или столкнувшегося с ошибкой, а также к точному времени наступления события. Это позволяет в дальнейшем выполнять анализ поведения системы, выявлять причины сбоев и обеспечивать аудит действий пользователей.

Программная архитектура серверной части, реализованная на платформе ASP.NET Core, в графическом виде представлена на рисунке 5.1. Диаграмма классов отражает ключевые сущности предметной области (User, Subject, Test, Task, Attempt, UserAnswer), сервисный слой с классами AuthService, TestService, AttemptService и UserAnswerRepository, интерфейсы репозитория для доступа к данным (IUserRepository, ITestRepository, IAttemptRepository), а также контекст базы данных ApplicationDbContext, агрегирующий коллекции DbSet для всех сущностей и обеспечивающий сохранение изменений через метод SaveChanges().

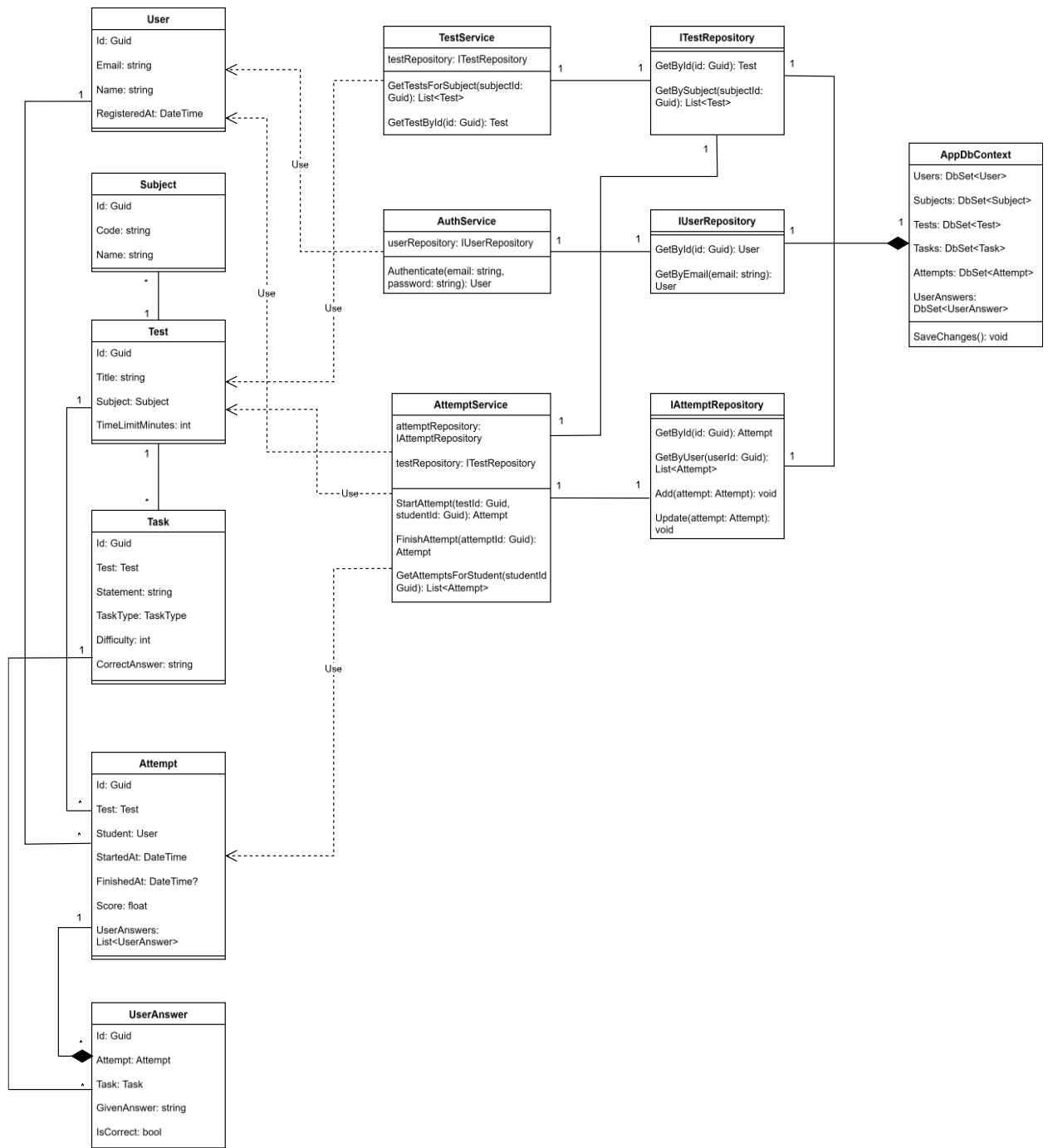


Рисунок 5.1 — Диаграмма классов серверной части (ASP.NET Core)

5.3 Структура мобильного приложения

Мобильное приложение реализовано на языке Kotlin для операционной системы Android с использованием архитектурного паттерна MVVM. В приложении выделены следующие уровни: пользовательский интерфейс, уровень управления состоянием и уровень работы с данными.

Пользовательский интерфейс реализован с использованием Jetpack Compose и включать экраны:

- экран регистрации и входа, обеспечивающий ввод данных и взаимодействие с сервером для получения токена авторизации;

- главный экран с выбором предмета и типа теста;

- экран списка тестов прошлых лет с возможностью фильтрации по предмету и году;

- экран генерации смешанного теста с указанием количества заданий по категориям и уровням сложности;

- экран назначенных тестов, отображающий задания, полученные от преподавателя;

- экран прохождения теста, включающий таймер, навигацию по вопросам и ввод ответов;

- экран результатов с указанием количества правильных и неправильных ответов, общей оценки и времени выполнения;

- экран истории решений, содержащий список попыток и возможность просмотра подробных результатов каждой;

- экран статистики, отображающий динамику подготовки, распределение ошибок по темам и рекомендации к повторению;

- экран уведомлений, отображающий список событий, полученных от сервера.

Уровень управления состоянием реализован с использованием ViewModel и обеспечивать хранение состояния экранов, обработку бизнес-логики на клиенте, а также вызовы к API. Каждое основное направление функциональности (аутентификация, тесты, попытки, статистика, уведомления) имеет собственную ViewModel.

Уровень работы с данными включает:

- сетевой модуль для взаимодействия с серверной частью через Retrofit, с поддержкой авторизации по JWT-токену;

- локальное хранилище на базе Room/SQLite для кэширования тестов, сохранения незавершённых попыток и работы в офлайн-режиме;

- модуль синхронизации, обеспечивающий отправку данных на сервер после восстановления сети.

Приложение поддерживает разграничение интерфейса в зависимости от роли пользователя. Для роли «ученик» доступны функции прохождения тестов, просмотра статистики и получения рекомендаций. Для роли «преподаватель» дополнительно доступны функции просмотра статистики прикреплённых учеников и их истории решений.

Архитектура приложения обеспечивает модульность. Добавление новых экранов и расширение функционала не должно требовать изменений в существующих модулях. Взаимодействие между компонентами осуществляется через чётко определённые интерфейсы.

Приложение поддерживает обработку ошибок: сетевых сбоев, истечения срока действия токена, недоступности сервера. Все ошибки отображаются пользователю в виде уведомлений или диалогов.

5.4 Структура веб-инструмента преподавателя

Веб-инструмент преподавателя реализован на платформе Django с использованием Django Templates. Интерфейс доступен в современных браузерах (Google Chrome, Mozilla Firefox, Microsoft Edge) и поддерживает минимальное разрешение экрана 1366×768 пикселей.

В системе реализованы следующие разделы интерфейса:

1 Раздел авторизации – форма входа по электронной почте и паролю с проверкой учётных данных через серверную часть.

2 Раздел управления тестами – список созданных тестов с возможностью сортировки, поиска и фильтрации по предмету, дате создания и статусу публикации.

3 Раздел создания и редактирования теста – форма для добавления заданий из банка и создания новых вопросов. Форма должна включать поля для формулировки задания, типа задания, уровня сложности, правильного ответа и пояснения. Преподаватель должен иметь возможность изменять порядок заданий, указывать лимит времени и число попыток для теста.

Веб-инструмент должен обеспечивать валидацию всех вводимых данных на клиентской и серверной стороне. Ошибки валидации должны отображаться в интерфейсе пользователя.

Все взаимодействие с сервером выполняется через REST API. Запросы передаются по протоколу HTTPS с использованием JWT-токенов для авторизации. Данные о создании, редактировании и назначении тестов сохраняются в базе данных через серверную часть.

Веб-инструмент использует адаптивные шаблоны. Интерфейс должен корректно отображаться на различных диагоналях экрана без потери функциональности. Элементы управления сгруппированы по назначению и обеспечивают минимальное количество действий для выполнения базовых операций.

Для предотвращения потери данных при создании тестов реализован механизм сохранения черновиков. Черновик теста должен оставаться в системе до момента публикации или удаления преподавателем.

Веб-инструмент должен обеспечивать разграничение доступа. Все функции управления тестами и учениками доступны только авторизованным пользователям с ролью преподавателя.

Структура классов мобильного приложения, спроектированного по архитектурному паттерну MVVM на языке Kotlin, изображена на рисунке 5.4. Диаграмма включает базовые компоненты (BaseViewModel с состоянием загрузки, Screen с маршрутизацией, NavigationController), конкретные модели представления (HomeViewModel и TestViewModel для основных экранов, SettingsViewModel для настроек), экран прохождения теста TestScreen, модели данных предметной области (Test, Question с вариантами ответов, Attempt, UserAnswer с логикой проверки правильности), а также репозиторий

TestRepository, обеспечивающий взаимодействие с удалённым API-сервисом и локальным DAO-слоем.

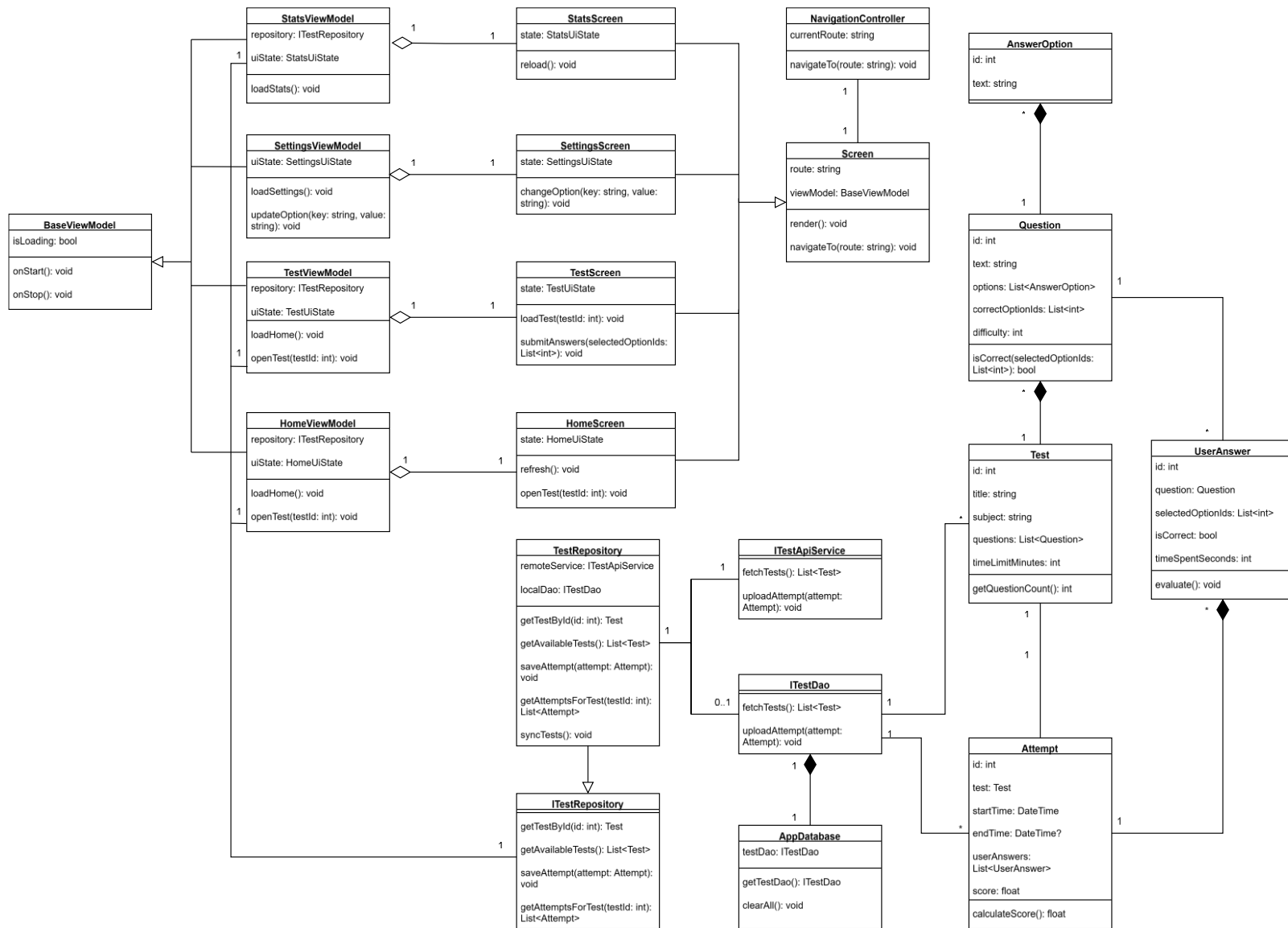


Рисунок 5.4 — Диаграмма классов мобильного приложения (Kotlin, MVVM)

5.5 Структура административной панели

Административная панель реализована с использованием встроенного инструмента Django Admin и предназначена для управления основными сущностями системы. Доступ к панели предоставляется только пользователям с ролью администратора или разработчика. Авторизация выполняется по имени пользователя и паролю с последующей проверкой прав доступа.

В административной панели реализованы следующие функции:

- управление пользователями: создание, редактирование, блокировка и удаление учётных записей, изменение ролей;
- управление предметами: добавление и редактирование наименований предметов, их кодов и статусов активности;
- управление темами: добавление, редактирование и удаление тем, привязка тем к предметам, настройка иерархии тем;
- управление заданиями: создание, редактирование и удаление заданий, изменение формулировки, правильного ответа, уровня сложности, привязки к предмету и теме;
- управление тестами: просмотр и редактирование тестов прошлых лет и пользовательских тестов, изменение их параметров, статуса публикации и состава;
- просмотр назначений тестов: проверка связей «преподаватель—ученик» и списка назначенных тестов;
- просмотр и редактирование справочников (например, источников экзаменационных вариантов).

Панель предоставляет стандартный интерфейс Django Admin с возможностью поиска, фильтрации и сортировки записей. Для всех сущностей реализованы формы редактирования с валидацией данных.

Все изменения, выполняемые через административную панель, фиксируются в журнале событий. В журнале хранятся сведения о пользователе, дате и времени операции, изменённых сущностях и типе действия (создание, редактирование, удаление).

Административная панель обеспечивает поддержку массовых операций: удаление нескольких записей, активация и деактивация выбранных элементов.

5.6 Поддерживаемые платформы и окружения

Мобильное приложение поддерживает операционную систему Android версии не ниже 8.0 (API level 26). Приложение должно корректно функционировать на устройствах с архитектурой процессора ARM и x86, с минимальным объёмом оперативной памяти 2 ГБ и минимальным разрешением экрана 720×1280 пикселей. Приложение должно быть оптимизировано для работы как на смартфонах, так и на планшетах.

Серверная часть поддерживает развёртывание на операционных

системах Windows Server 2019 и выше и Ubuntu Linux 22.04 и выше [15]. Допускается использование других совместимых дистрибутивов Linux. Для удобства развёртывания и сопровождения серверная часть поддерживает работу в контейнерах Docker.

База данных реализована в СУБД PostgreSQL версии 15 или выше. База данных поддерживает развёртывание на выделенных серверах, в контейнерах Docker, а также в облачных средах (например, Amazon RDS или Microsoft Azure Database for PostgreSQL).

Веб-инструмент для преподавателей и административная панель поддерживают работу в современных браузерах: Google Chrome (версии не ниже 110), Mozilla Firefox (версии не ниже 110) и Microsoft Edge (версии не ниже 110). Интерфейс корректно отображается при минимальном разрешении экрана 1366×768 и выше.

Для системы определены следующие минимальные аппаратные требования для развёртывания:

- сервер приложений: 4 ядра CPU, 8 ГБ оперативной памяти, 50 ГБ дискового пространства;
- сервер базы данных: 4 ядра CPU, 8 ГБ оперативной памяти, 100 ГБ дискового пространства;
- возможность горизонтального масштабирования серверной части при возрастании нагрузки.

Все компоненты программного комплекса поддерживают работу в сетевой среде с использованием протокола HTTPS и шифрования TLS версии 1.3 и выше.

6 Экономическое обоснование разработки и реализации на массовом рынке программного комплекса для подготовки к централизованному тестированию и централизованному экзамену

6.1 Характеристика программного средства

Разрабатываемый программный комплекс представляет собой цифровой сервис для подготовки к централизованному экзамену (ЦЭ) и централизованному тестированию (ЦТ). Система объединяет функции самостоятельной подготовки учащегося и инструменты контроля со стороны преподавателя в единой образовательной среде.

Функционально сервис включает:

- структурированную базу тестовых заданий прошлых лет с возможностью поиска по темам;
- генерацию индивидуальных тестов (смешанных, по выбранным темам, случайных);
- автоматический сбор статистики решений с детализацией по темам и типам ошибок;
- формирование персонализированных рекомендаций по повторению материала;
- личный кабинет преподавателя с возможностью создания собственных тестов, привязки учеников, мониторинга их прогресса и генерации аналитических отчётов.

Целевая аудитория – три основные группы. Учащиеся (выпускники школ, абитуриенты) получают инструмент для системной самоподготовки. Преподаватели и репетиторы используют сервис для создания тестов, контроля успеваемости и взаимодействия с учениками. Образовательные учреждения (школы, гимназии, центры) внедряют комплекс для организации пробных экзаменов и внутреннего мониторинга знаний. Общая численность потенциальных пользователей в Беларуси составляет более 68 тыс. учащихся и 28,5 тыс. преподавателей, что обеспечивает ёмкий рынок.

Программный комплекс ориентирован на три категории: учащихся, преподавателей и образовательные учреждения. Монетизация построена по гибридной модели Freemium с платными подписками, где бесплатный базовый функционал дополнен расширенными возможностями (глубокая аналитика, генерация тестов, работа с учениками), а для преподавателей предусмотрена градация тарифов по количеству учеников. Продвижение осуществляется через магазины приложений, таргетированную и контекстную рекламу, партнёрства с образовательными центрами, профильные мероприятия и прямые продажи школам. Реализация продукта обеспечивает стабильный доход от подписок, масштабируемость бизнеса и рост доли компании на рынке

цифровых образовательных сервисов.

6.2 Расчет инвестиций в разработку программного средства

Планируемый срок разработки MVP (Minimum Viable Product) программного комплекса составит 6 месяцев. Данный срок включает этапы проектирования архитектуры, разработки серверной части, мобильного приложения, веб-инструмента для преподавателей, тестирования и подготовки к опытной эксплуатации.

Соотношение премии к зарплате принято 50%.

Расчетная норма рабочего времени при пятидневной рабочей неделе составляет 2016 часов в год. (4.1).

$$Z_o = K_{пр} \cdot \sum_{i=1}^n Z_{чи} \cdot t_i, \quad (4.1)$$

где $K_{пр}$ – коэффициент премий и иных стимулирующих выплат;
 n – количество исполнителей, занятых разработкой конкретного ПО, шт;
 $Z_{чи}$ – часовой оклад исполнителя i -й категории, р;
 t_i – трудоемкость работ, выполняемых исполнителем i -й категории, ч.
 Расчет основной заработной платы представлен в таблице 4.1.

Таблица 4.1 – Расчет основной заработной платы команды разработчиков программного средства

Категория исполнителя	Месячный оклад, р.	Часовой оклад, р.	Трудоемкость работ, ч	Итого, р.
Бизнес аналитик	4000	23,81	150	3571,50
Backend-разработчик (ASP.NET Core)	5 500	32,74	450	14733,00
Android-разработчик (Kotlin)	5 000	29,76	450	13392,00
Fullstack-разработчик (Django/Python)	3 500	20,83	300	6249,00
Тестировщик (QA)	2 300	13,69	200	2738,00
Итого				40683,50
Премия и иные стимулирующие выплаты (50%)				20341,75
Всего затрат на основную заработную плату разработчиков				61025,25

Расчет общей суммы инвестиций (затрат) на разработку программного средства осуществляется по методике, представленной в таблице 4.2.

Таблица 4.2 – Расчет инвестиций (затрат) на разработку программного

средства

Наименование статьи затрат	Формула/таблица для расчета, р.	Сумма, р.
1. Основная заработная плата разработчиков	Формула (4.1), таблица 4.1	61025,25
2. Дополнительная заработная плата разработчиков	$З_д = \frac{З_о \cdot Н_д}{100} = \frac{61025,25 \cdot 10}{100} = 6102,53, \quad (4.2)$ где $Н_д$ – норматив дополнительной заработной платы, (10%)	6102,53
3. Отчисления на социальные нужды	$P_{соц} = \frac{(З_о + З_д) \cdot Н_{соц}}{100}$ $= \frac{(61025,25 + 6102,53) \cdot 34,6}{100} = 23230,21, \quad (4.3)$ где $Н_{соц}$ – ставка отчислений в ФСЗН и Белгосстрах (в соответствии с действующим законодательством – 34,6%)	23230,21
4. Прочие расходы	$P_{пр} = \frac{З_о \cdot Н_{пр}}{100}$ $= \frac{61025,25 \cdot 30}{100} = 18307,58, \quad (4.4)$ где $Н_{пр}$ – норматив прочих расходов, (30%)	18307,58
5. Расходы на реализацию	$P_p = \frac{З_о \cdot Н_p}{100}$ $= \frac{61025,25 \cdot 5}{100} = 3051,26, \quad (4.5)$ где $Н_p$ – норматив расходов на реализацию, (5 %)	3051,26
6. Общая сумма инвестиций (затрат) на разработку	$З_p = З_о + З_д + P_{соц} + P_{пр} + P_p$ $= 61025,25 + 6102,53 + 23230,21 + 18307,58 + 3051,26 = 111716,83, \quad (4.6)$	111716,83

Таким образом полная сумма затрат на разработку программного обеспечения находится путем суммирования всех рассчитанных статей и

составляет 111716,83 рублей.

6.3 Расчет экономического эффекта от реализации программного средства на рынке

Экономический эффект организации-разработчика программного средства представляет собой прирост чистой прибыли от его продажи на рынке потребителям, величина которого зависит от объема продаж, цены реализации и затрат на разработку программного средства.

По данным Республиканского института контроля знаний, в 2025 году в централизованном тестировании приняли участие 58491 абитуриент. Централизованный экзамен сдавали 56700 выпускников 11-х классов.

С учётом того, что подготовку к ЦТ начинают уже в 10-м классе, и доля десятиклассников в общей массе готовящихся составляет 17,5% (по экспертным оценкам), общая численность целевой аудитории учащихся рассчитывается следующим образом:

- учащиеся 11-х классов: 56700 чел.
- учащиеся 10-х классов, готовящиеся к ЦТ: $56700 \times 0,175 = 9923$ чел.
- выпускники прошлых лет и учащиеся колледжей: $58491 - 56700 = 1791$ чел.

Общая численность целевой аудитории учащихся: $56700 + 9923 + 1791 \approx 68400$ человек.

Количество преподавателей, потенциально заинтересованных в использовании системы: по данным Министерства образования, в сфере образования Беларуси работают 114 000 педагогов. Учитывая, что не все преподаватели ведут выпускные классы и готовят к ЦТ, по имеющимся оценкам доля целевых преподавателей составляет 25% от общего числа, то есть 28 500 человек.

Принимая во внимание наличие бесплатного тарифа (Freemium) и конкуренцию на рынке (Adukar, TurboCT, 0101CT), прогнозируемый уровень проникновения платной подписки в первый год составит:

- среди учащихся: 1,5% от целевой аудитории
- среди преподавателей: 1,0% от целевой аудитории

Таким образом, прогнозируемое количество платящих пользователей в первый год:

- учащиеся: $68400 \times 0,015 = 1026$ чел.
- преподаватели: $28500 \times 0,01 = 285$ чел.

Общее число платящих пользователей $N = 1311$ человек.

На основе анализа цен на образовательные услуги в Беларуси (курсы подготовки при вузах стоят 350–600 руб. за сезон), а также с учётом разработанной тарифной сетки, установим следующие цены:

1. Подписка «Ученик» (доступ к полной аналитике, рекомендациям, неограниченным тестам): 25 руб./мес. (годовая подписка – 250 руб. со скидкой)

2. Подписка «Преподаватель» (создание тестов, привязка учеников, аналитика по группе): 40 руб./мес. (годовая – 400 руб.)

Средневзвешенная стоимость подписки (ARPU) с учётом структуры пользователей:

$$ARPU = \frac{(1026 \times 250 + 285 \times 400)}{1311} \approx 283 \text{ р.}$$

Прирост чистой прибыли, полученной разработчиком от реализации программного средства на рынке, рассчитывается по формуле:

$$\Delta\Pi_{\text{ч}}^{\text{р}} = (\Pi_{\text{отп}} \cdot N - \text{НДС}) \cdot P_{\text{пр}} \cdot \left(1 - \frac{H_{\text{п}}}{100}\right), \quad (4.7)$$

где $\Pi_{\text{отп}}$ – отпускная цена подписки программного средства, р.;

N – количество подписки программного средства, реализуемое за год, шт.;

НДС – сумма налога на добавленную стоимость, р.;

$P_{\text{пр}}$ – рентабельность продаж (принята на уровне 30%);

$H_{\text{п}}$ – ставка налога на прибыль согласно действующему законодательству (20%).

Налог на добавленную стоимость определяется по формуле:

$$\text{НДС} = \frac{\Pi_{\text{отп}} \cdot N \cdot H_{\text{д.с}}}{100\% + H_{\text{д.с}}}, \quad (4.8)$$

где $H_{\text{д.с}}$ – ставка налога на добавленную стоимость в соответствии с действующим законодательством (20%).

Рассчитаем НДС с помощью формулы (4.8):

$$\text{НДС} = \frac{283 \cdot 1311 \cdot 20}{120} = 61750 \text{ р.}, \quad (4.9)$$

Годовая выручка без НДС:

$$283 \cdot 1311 - 61317 = 308750 \text{ р.}$$

Прирост чистой прибыли, полученной разработчиком от реализации программного средства на рынке, рассчитаем по формуле (4.7):

$$\Delta\Pi_{\text{ч}}^{\text{р}} = 308750 \cdot 0,3 \cdot \left(1 - \frac{20}{100}\right) = 74100 \text{ р.}, \quad (4.10)$$

Таким образом, прирост чистой прибыли, полученной разработчиком от реализации программного средства в первый год, составляет 147160 рублей.

6.4 Расчет показателей экономической эффективности разработки и реализации программного средства на рынке

Экономическая эффективность инвестиций в разработку программного средства и его реализацию на рынке осуществляется на основе расчета и оценки показателей эффективности инвестиций по методике, представленной в таблице 4.3.

Исходные данные для расчета:

- инвестиции в разработку – 111717,00 рублей;
- годовой экономический эффект – 74100,00 рублей;
- норма дисконта – 9,75% (принята равной ставке рефинансирования Национального банка Республики Беларусь на 2026 год);
- расчетный период – 2 года.

Подсчет чистого дисконтированного дохода осуществим в табличной форме. Расчет эффективности инвестиций показан в таблице 4.5.

Таблица 4.3 – Расчет эффективности инвестиций (затрат) в реализацию проектного решения

Показатель	Значение по годам расчетного периода	
	07.2026 – 07.2027	07.2027 – 07.2028
Результат		
1 Прирост чистой прибыли, р.	74100,00	74100,00
2 Дисконтированный результат, р.	74100,00	67518,95
Затраты		
3 Инвестиции в реализацию проектного решения, р.	111716,83	0
4 Дисконтированные инвестиции, р.	111716,83	0
5 Чистый дисконтированный доход по годам, р.	74100,00	67518,95
6 Чистый дисконтированный доход нарастающим итогом, р.	-37616,83	29901,17
7 Коэффициент дисконтирования, доли единицы	1,0000	0,9112

Чистый дисконтированный доход за 2 года, полученный по формуле (4.11) составляет ЧДД(NPV) = 29901,17 рубля, что удовлетворяет целевому значению «> 0».

$$\text{ЧДД(NPV)} = \sum_{t=1}^n \Delta\Pi_{\text{чт}} \cdot \alpha_t - \sum_{t=1}^n Z_t \cdot \alpha_t, \quad (4.11)$$

где α_t – коэффициент дисконтирования, рассчитанный для года t

Также табличный расчет по формуле (4.12) позволяет получить значение динамического срока окупаемости инвестиций. DPP = 2 г, так как именно во 2-й год рассчитанное значение удовлетворяет целевому «≤ 3 года».

$$\text{DPP} = n, \text{ при котором } \sum_{t=1}^n \Delta\Pi_{\text{чт}} \cdot \frac{1}{(1+d)^{t-t_p}} \geq \sum_{t=1}^n Z_t \cdot \frac{1}{(1+d)^{t-t_p}}, \quad (4.12)$$

Рассчитаем индекс доходности инвестиций за весь расчётный период (2 года):

$$\text{ИД(PI)} = \frac{\sum_{t=1}^n \Delta\Pi_{\text{чт}} \cdot \alpha_t}{\sum_{t=1}^n Z_t \cdot \alpha_t} = \frac{74100,00 + 67518,00}{111716,83} = 1,27, \quad (4.13)$$

Коэффициент дисконтирования α_t , получаемый для года t , рассчитывается по формуле:

$$\alpha_t = \frac{1}{(1+d)^{t-t_p}}, \quad (4.14)$$

где d – требуемая норма дисконта, доли единицы;

t – порядковый номер года, доходы и затраты которого приводятся к расчетному году;

t_p – расчетный год, к которому приводятся доходы и инвестиционные затраты ($t_p = 1$).

Значение, полученное в формуле (4.13), соответствует целевому «> 1». По формуле (4.11) получим простой срок окупаемости инвестиций:

$$T_{\text{ок}}(\text{PP}) = \frac{\sum_{t=1}^n Z_t}{\frac{1}{n} \cdot \sum_{t=1}^n \Delta\Pi_{\text{чт}}} = \frac{111716,83}{74100,00} \approx 1,51 \text{ года } (\approx 18 \text{ месяцев}), \quad (4.15)$$

Где

- n – расчетный период, лет
- Z_t – затраты (инвестиции) в году t , р.
- $\Delta\Pi_{qt}$ – прирост чистой прибыли в году t в результате реализации проекта, р.

Значение простого срока окупаемости инвестиций, полученное в формуле (4.7), соответствует целевому « ≤ 3 года». Получим и сравним с целевым значением среднюю норму прибыли/рентабельности инвестиций:

$$P_{\text{и}}(\text{ARR}) = \frac{\frac{1}{n} \cdot \sum_{t=1}^n \Delta\Pi_{qt}}{\sum_{t=1}^n Z_t} \cdot 100\% = \frac{74100,00}{111716,83} \cdot 100\% = 66,3\%, \quad (4.16)$$

Полученное значение средней нормы прибыли/рентабельности инвестиций, полученное в формуле (4.8), значительно превышает среднюю процентную ставку в национальной валюте для юридических лиц на срок свыше 1 года на 2026 год (10,65%).

Данные, полученные за два года работы продукта, свидетельствуют о том, что инвестиции окупаются уже на втором году реализации проекта. Показатели доходности подтверждают привлекательность вложений, а средняя норма рентабельности инвестиций значительно превышает норму дисконта. Таким образом, разработка и реализация на рынке программного комплекса являются экономически целесообразными.

ЗАКЛЮЧЕНИЕ

В ходе выполнения проекта был проведён анализ существующих цифровых решений, ориентированных на подготовку учащихся к централизованному тестированию и централизованному экзамену. Рассмотренные онлайн-платформы и мобильные приложения обладают рядом достоинств, однако не обеспечивают комплексного подхода к подготовке. Сформулированные на основании анализа цели и задачи были полностью реализованы в готовом программном комплексе.

В разработанной системе реализованы все зафиксированные в техническом задании функциональные и нефункциональные требования. Программный комплекс обеспечивает поддержку всех ролей пользователей, включает механизмы генерации тестов различных типов, формирует детальную статистику и персонализированные рекомендации, содержит систему уведомлений и предоставляет преподавателю полноценные инструменты для работы с учениками. Обеспечены требования к безопасности, интерфейсам и производительности, что гарантирует надёжность и устойчивость системы в условиях реальной эксплуатации.

Архитектура комплекса построена на основе многослойной клиент–серверной модели. В её состав входят мобильное приложение для операционной системы Android, серверная часть на платформе ASP.NET Core, веб-инструмент преподавателя на базе Django Templates и административная панель на основе Django Admin. Для хранения данных задействована СУБД PostgreSQL, что обеспечивает отказоустойчивость и возможность дальнейшего масштабирования. В процессе разработки были созданы логическая и физическая модели базы данных, описаны основные сущности и связи, реализованы правила целостности и индексации, которые эффективно функционируют в текущей версии продукта.

Программная реализация полностью соответствует утверждённым требованиям к структуре серверной части, мобильного приложения, веб-инструмента и административной панели. Проведённое экономическое обоснование подтвердило целесообразность разработки. В ходе проекта были подробно описаны используемые средства и инструменты, а также определены минимальные условия развертывания и перечень поддерживаемых платформ.

Практическая значимость разработанного продукта заключается в создании специализированного цифрового инструмента, ориентированного на специфику белорусской системы образования. Комплекс успешно объединяет функции самостоятельной подготовки и контроля со стороны преподавателя, что позволяет повысить эффективность учебного процесса, сделать его более гибким и доступным. Реализованный проект обеспечивает учащимся возможность систематизировать знания и отслеживать динамику подготовки в реальном времени, а преподавателям — использовать инструменты для углублённого анализа результатов и эффективной организации

взаимодействия с учениками.

СПИСОК ЛИТЕРАТУРНЫХ ИСТОЧНИКОВ

- [1] PostgreSQL 15 Documentation [Электронный ресурс]. – Режим доступа: <https://www.postgresql.org/docs/15/index.html> – Дата доступа: 05.04.2025.
- [2] .NET Documentation [Электронный ресурс]. – Режим доступа: <https://learn.microsoft.com/ru-ru/dotnet/> – Дата доступа: 05.04.2025.
- [3] Современные веб-приложения на ASP.NET Core [Электронный ресурс]. – Режим доступа: <https://learn.microsoft.com/ru-ru/dotnet/architecture/modern-web-apps-azure/> – Дата доступа: 05.04.2025.
- [4] Npgsql Documentation [Электронный ресурс]. – Режим доступа: <https://www.npgsql.org/doc/index.html> – Дата доступа: 05.04.2025.
- [5] Руководство. Создание веб-API на основе контроллера с помощью ASP.NET Core [Электронный ресурс]. – Режим доступа: <https://learn.microsoft.com/ru-ru/aspnet/core/tutorials/first-web-api?view=aspnetcore-9.0&tabs=visual-studio> – Дата доступа: 05.04.2025.
- [6] Introduction to Swagger and OpenAPI [Электронный ресурс]. – Режим доступа: <https://swagger.io/docs/specification/about/> – Дата доступа: 05.04.2025.
- [7] JSON Web Tokens Introduction [Электронный ресурс]. – Режим доступа: <https://jwt.io/introduction> – Дата доступа: 05.04.2025.
- [8] OAuth 2.0 [Электронный ресурс]. – Режим доступа: <https://oauth.net/2/> – Дата доступа: 05.04.2025.
- [9] What is OpenID Connect [Электронный ресурс]. – Режим доступа: <https://openid.net/developers/how-connect-works/> – Дата доступа: 05.04.2025.
- [10] Transport Layer Security (TLS) [Электронный ресурс]. – Режим доступа: <https://www.cloudflare.com/learning/ssl/what-is-ssl/> – Дата доступа: 05.04.2025.
- [11] PostgreSQL Documentation: JSON Types [Электронный ресурс]. – Режим доступа: <https://www.postgresql.org/docs/15/datatype-json.html> – Дата доступа: 05.04.2025.
- [12] PostgreSQL Documentation: pg_trgm [Электронный ресурс]. – Режим доступа: <https://www.postgresql.org/docs/15/pgtrgm.html> – Дата доступа: 05.04.2025.
- [13] PostgreSQL Documentation: PL/pgSQL [Электронный ресурс]. – Режим доступа: <https://www.postgresql.org/docs/15/plpgsql.html> – Дата доступа: 05.04.2025.
- [14] Logging in .NET [Электронный ресурс]. – Режим доступа: <https://learn.microsoft.com/ru-ru/dotnet/core/extensions/logging> – Дата доступа: 05.04.2025.
- [15] Руководство по публикации приложений .NET [Электронный ресурс]. – Режим доступа: <https://learn.microsoft.com/ru-ru/dotnet/core/deploying/> – Дата доступа: 05.04.2025.

ПРИЛОЖЕНИЕ А

Справка о проверке на заимствования

Проверить текст на уникальность

Длина текста: 2982 (без пробелов: 2629)

В основу схемы положена третья нормальная форма: справочники отделены от фактов, связи многие-ко-многим вынесены в отдельные таблицы, критичные зависимости контролируются внешними ключами и триггерами. Однако для достижения приемлемой производительности и удобства сопровождения применено несколько осознанных отступлений.

Справочники и иерархии. Сущности «Предмет», «Тема», «Источник экзамена» вынесены в отдельные таблицы, что исключает дублирование названий и кодов в задачных или тестовых записях. Темы образуют дерево через `topic_parent_id`, поэтому их структура до третьей нормальной формы: каждая запись зависит только от первичного ключа и ссылки на родителя. При необходимости скрыть целый предмет или ветку тем достаточно изменить признак активности, не затрагивая историю попыток.

Связи многие-ко-многим. Таблицы `teacher_student` и `test_task` фиксируют отношения между преподавателями и учениками, тестами и заданиями. Они содержат только ключи и служебные поля (статус связи, дата установления, порядок, вес), благодаря чему модель остаётся чистой. При удалении пользователя или теста каскадные правила гарантируют отсутствие «висячих» ссылок, а бизнес-логика (ограничение количества попыток, проверка статусов) реализуется триггерами и процедурами.

Денормализованный слой статистики. Таблица `user_stats` хранит агрегаты по предметам и темам: число попыток, количество верных ответов, средний балл, среднее время, дату последнего обновления. Она обновляется процедурами PostgreSQL сразу после завершения попытки, поэтому REST API и консольный клиент получают готовые показатели без тяжёлых JOIN. Источником данных остаются нормализованные таблицы `attempt`, `user_answer`, что позволяет при необходимости полностью пересчитать агрегаты.

Хранение результатов. Подробные сведения о попытках нормализованы: попытка (`attempt`) и ответ (`user_answer`) разделены, что даёт возможность анализировать ошибки на уровне вопросов. При этом полезная нагрузка рекомендации (`recommendation`) сохраняется в JSONB, потому что их структура зависит от типа события. Такой подход уменьшает количество вспомогательных таблиц, а корректность содержимого контролируют триггеры и проверочные процедуры.

Использование JSONB в заданиях. Поля `correct_answer`, `given_answer` используют JSONB для хранения множественных ответов, сопоставлений, шаблонов проверки свободного текста. Это отступление от классической атомарности, но оно оправдано: каждое задание может требовать разных структур, а одновременное ведение десятков вспомогательных таблиц усложнило бы сопровождение. Целостность обеспечивается хранимыми процедурами, валидирующими структуру JSON в зависимости от `task_type`.

Таким образом, схема сочетает нормализованные справочники и связи с аккуратно контролируруемыми денормализованными областями: агрегатами статистики и JSONB-полями. Это обеспечивает баланс между гибкостью модели предметной области, скоростью работы REST API и удобством сопровождения базы данных в учебной и эксплуатационной среде.

Уникальность текста: 100.0%

Править этот текст

Новая проверка

Рисунок 1 – Результат проверки на заимствование

ПРИЛОЖЕНИЕ Б

Листинг программного кода

```
CREATE TABLE IF NOT EXISTS role
(
    id          BIGSERIAL PRIMARY KEY,
    role_name   VARCHAR(100) NOT NULL UNIQUE,
    description TEXT,
    created_at  TIMESTAMPTZ  NOT NULL DEFAULT NOW(),
    updated_at  TIMESTAMPTZ  NOT NULL DEFAULT NOW()
);

CREATE TABLE IF NOT EXISTS user_account
(
    id          BIGSERIAL PRIMARY KEY,
    user_name   VARCHAR(150) NOT NULL,
    email       VARCHAR(254) NOT NULL UNIQUE,
    password_hash VARCHAR(256) NOT NULL,
    role_type_id BIGINT      NOT NULL REFERENCES role (id),
    last_login_at TIMESTAMPTZ,
    created_at  TIMESTAMPTZ  NOT NULL DEFAULT NOW(),
    updated_at  TIMESTAMPTZ  NOT NULL DEFAULT NOW()
);

CREATE TABLE IF NOT EXISTS subject
(
    id          BIGSERIAL PRIMARY KEY,
    subject_code VARCHAR(32)  NOT NULL UNIQUE,
    subject_name VARCHAR(150) NOT NULL,
    is_active   BOOLEAN      NOT NULL DEFAULT TRUE,
    created_at  TIMESTAMPTZ  NOT NULL DEFAULT NOW(),
    updated_at  TIMESTAMPTZ  NOT NULL DEFAULT NOW()
);

CREATE TABLE IF NOT EXISTS topic
(
    id          BIGSERIAL PRIMARY KEY,
    subject_id  BIGINT      NOT NULL REFERENCES subject (id),
    topic_name  VARCHAR(150) NOT NULL,
    topic_code  VARCHAR(64),
    topic_parent_id BIGINT REFERENCES topic (id),
    is_active   BOOLEAN      NOT NULL DEFAULT TRUE,
    created_at  TIMESTAMPTZ  NOT NULL DEFAULT NOW(),
    updated_at  TIMESTAMPTZ  NOT NULL DEFAULT NOW()
);

CREATE TABLE IF NOT EXISTS exam_source
(
    id          BIGSERIAL PRIMARY KEY,
    year        INTEGER      NOT NULL,
    variant_number INTEGER,
    issuer      VARCHAR(150),
```

```

        notes          TEXT,
        created_at     TIMESTAMPTZ    NOT NULL DEFAULT NOW(),
        updated_at     TIMESTAMPTZ    NOT NULL DEFAULT NOW()
    );

-- Task/test related tables
CREATE TABLE IF NOT EXISTS task_item
(
    id                BIGSERIAL PRIMARY KEY,
    subject_id       BIGINT          NOT NULL REFERENCES subject (id),
    topic_id         BIGINT REFERENCES topic (id),
    exam_source_id   BIGINT REFERENCES exam_source (id),
    task_type        VARCHAR(64) NOT NULL,
    difficulty        SMALLINT       NOT NULL,
    statement         TEXT            NOT NULL,
    correct_answer    JSONB          NOT NULL,
    explanation       TEXT,
    is_active         BOOLEAN         NOT NULL DEFAULT TRUE,
    created_at        TIMESTAMPTZ    NOT NULL DEFAULT NOW(),
    updated_at        TIMESTAMPTZ    NOT NULL DEFAULT NOW()
);

CREATE TABLE IF NOT EXISTS test
(
    id                BIGSERIAL PRIMARY KEY,
    subject_id       BIGINT          NOT NULL REFERENCES subject (id),
    test_kind        VARCHAR(64)   NOT NULL,
    title            VARCHAR(200) NOT NULL,
    author_id        BIGINT REFERENCES user_account (id),
    time_limit_sec   INTEGER,
    attempts_allowed SMALLINT,
    mode             VARCHAR(50),
    is_published     BOOLEAN         NOT NULL DEFAULT FALSE,
    is_public        BOOLEAN         NOT NULL DEFAULT FALSE,
    is_state_archive BOOLEAN         NOT NULL DEFAULT FALSE,
    created_at       TIMESTAMPTZ    NOT NULL DEFAULT NOW(),
    updated_at       TIMESTAMPTZ    NOT NULL DEFAULT NOW()
);

CREATE TABLE IF NOT EXISTS test_task
(
    id                BIGSERIAL PRIMARY KEY,
    test_id          BIGINT          NOT NULL REFERENCES test (id) ON DELETE
CASCADE,
    task_id          BIGINT          NOT NULL REFERENCES task_item (id),
    position         INTEGER         NOT NULL,
    weight           NUMERIC(6,2),
    created_at       TIMESTAMPTZ    NOT NULL DEFAULT NOW(),
    updated_at       TIMESTAMPTZ    NOT NULL DEFAULT NOW(),
    UNIQUE (test_id, task_id),
    UNIQUE (test_id, position)
);

CREATE TABLE IF NOT EXISTS teacher_student

```

```

(
    id                BIGSERIAL PRIMARY KEY,
    teacher_id        BIGINT          NOT NULL REFERENCES user_account
(id),
    student_id        BIGINT          NOT NULL REFERENCES user_account
(id),
    status            VARCHAR(50) NOT NULL,
    established_at     TIMESTAMPTZ,
    created_at        TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at        TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    UNIQUE (teacher_id, student_id)
);

CREATE TABLE IF NOT EXISTS invitation_code
(
    id                BIGSERIAL PRIMARY KEY,
    teacher_id        BIGINT          NOT NULL REFERENCES user_account
(id),
    code              VARCHAR(36) NOT NULL UNIQUE,    -- GUID format:
XXXXXXXXXX-XXXX-XXXX-XXXX-XXXXXXXXXXXX
    max_uses          INTEGER DEFAULT NULL,
    used_count        INTEGER DEFAULT 0,
    expires_at        TIMESTAMPTZ,
    status            VARCHAR(20) NOT NULL DEFAULT 'active',
    created_at        TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE IF NOT EXISTS test_student_access
(
    id                BIGSERIAL PRIMARY KEY,
    test_id           BIGINT          NOT NULL REFERENCES test (id) ON DELETE
CASCADE,
    student_id        BIGINT          NOT NULL REFERENCES user_account (id)
ON DELETE CASCADE,
    created_at        TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    UNIQUE (test_id, student_id)
);

CREATE INDEX IF NOT EXISTS idx_test_student_access_test_id ON
test_student_access (test_id);
CREATE INDEX IF NOT EXISTS idx_test_student_access_student_id ON
test_student_access (student_id);
CREATE TABLE IF NOT EXISTS task_item
(
    id                BIGSERIAL PRIMARY KEY,
    subject_id        BIGINT          NOT NULL REFERENCES subject (id),
    topic_id          BIGINT REFERENCES topic (id),
    task_type         VARCHAR(64) NOT NULL,
    difficulty        SMALLINT       NOT NULL,
    statement         TEXT           NOT NULL,
    correct_answer    JSONB          NOT NULL,
    explanation       TEXT,
    is_active         BOOLEAN        NOT NULL DEFAULT TRUE,
    created_at        TIMESTAMPTZ NOT NULL DEFAULT NOW(),

```



```
        updated_at      TIMESTAMPTZ NOT NULL DEFAULT NOW()  
    );
```

ПРИЛОЖЕНИЕ В

Графический интерфейс пользователя

ПРИЛОЖЕНИЕ Г
Use-case диаграмма программного средства

ПРИЛОЖЕНИЕ Д
Entity–relationship диаграмма программного средства

ПРИЛОЖЕНИЕ Е

Схема алгоритма обработки данных